科大讯飞 iFLYTEK | 星火大模型 SPARK DESK

用人工智能建设美好世界

CREATE A BETTER WORLD WITH ARTIFICIAL INTELLIGENCE

推免智策

大学生保研资格评估与升学规划智能体

复赛应用方案

2026 届科大讯飞星火杯大模型应用创新赛

更准确 · 更专业 · 更懂你

- 资格评估
- 院校匹配
- 风险诊断
- 成长规划
- 一站式解答



你的升学规划助手

- 数据解析中……
- 院校信息检索
- 综合评估分析
- 生成个性化建议

未来可期！



让每一位
努力的同学
看见更大的可能！

队伍名称：postgraduate recommendation assistant

团队成员：徐培智、步凡、韩林默、林臻阳、肖翔

二〇二六年

AI 赋 能 教 育　　智 慧 规 划 未 来

目录

一、项目背景 1
（一）现实需求与问题来源 1
（二）项目定位与参赛目标 1
二、项目阐述 4
（一）总体思路 4
（二）输入、处理与输出边界 4
（三）设计原则 5
三、产品功能 7
（一）学生资料读取与基础校验 7
（二）学校真实性与本科推免资格核验 7
（三）政策、培养方案和历史数据检索 8
（四）机会评估与风险诊断 8
（五）分年级升学规划 9
四、技术方案 13
（一）平台与节点编排 13
（二）三份核心脚本 13
1. 生成精准检索词 13
2. 学校核验结果标准化 14
3. 基础资料标准化 14
（三）学校核验算法逻辑 14
（四）联网检索策略与数据可信度 15
（五）大模型节点约束与输出稳定性 16
（六）统一输出与单结束节点 16
（七）异常处理与调试机制 17
五、运行效果 20
（一）正常场景 20
（二）大三场景 20

（三）异常与边界场景 20
（四）输出规范 21
（五）补充数据集的实测结果与回归价值 22
六、迭代计划 24
（一）近期迭代：稳定性与覆盖度 24
（二）中期迭代：政策索引与历史对比 25
（三）长期迭代：申请执行闭环 25
七、应用价值 26
（一）对学生的价值 26
（二）对高校升学指导的价值 26
（三）对同类智能体的参考价值 26
八、商业模式 28
（一）基础服务模式 28
（二）增值服务模式 28
（三）B 端合作可能 28
九、团队介绍 31
（一）团队信息 31
（二）迭代协作机制 31
十、技术深化说明 33
（一）工作流分层设计 33
（二）节点间变量与数据契约 34
（三）大模型与代码节点职责边界 36
（四）联网检索稳定性设计 37
十一、节点级实现说明 38
（一）开始与资料读取节点 38
（二）基础资料标准化脚本 38
（三）精准检索词生成脚本 39
（四）推免资料解析与数据口径控制 40

（五）学校核验结果标准化脚本 41
（六）分支、变量存储与唯一结束节点 41
（七）最终评估报告节点 43
（八）长报告参数与输出完整性 43
十二、测试与质量保障 45
（一）测试设计原则 45
（二）十组核心测试用例 45
（三）历史问题与回归测试 46
（四）补充数据集的故障样例转化 48
十三、部署复现与依赖说明 50
（一）完整平台依赖 50
（二）Python 与库依赖 51
十四、后续扩展路线 53
（一）短期迭代 53
（二）中长期扩展 53
十五、总结 54

一、项目背景

（一）现实需求与问题来源

推荐优秀应届本科毕业生免试攻读研究生（以下简称“推免”）并不是单一分数或单一排名决定的问题。不同高校、不同学院、不同届次的名额、硬性条件、综合成绩计算、竞赛科研加分办法和材料时间节点存在差异；同一所学校内部，不同学院也可能分别制定实施细则。学生实际进行规划时，通常需要在学校教务处、本科生院、学院官网、研究生招生网等多个页面之间反复检索，并手工判断“本科推荐资格”“研究生接收条件”“学院实施细则”等不同口径。这种信息分散和口径复杂的现状，导致学生在保研规划过程中面临极高的信息获取成本和决策风险。许多学生直到大三下学期才开始系统了解推免政策，此时已经错过了提升排名、积累竞赛科研成果的关键窗口期；还有部分学生因为对政策口径理解偏差，错误估计了自己的推免机会，最终在申请季陷入被动。因此，开发一个能够自动核验资格、统一数据口径、提供个性化规划的智能体工具，具有显著的现实需求和应用价值。

另一方面，大一、大二、大三学生的决策目标并不相同。低年级更关心“现在处于什么位置、未来两三年如何提升”，大三学生则需要在确认本校资格的同时，快速完成夏令营、预推免、目标院校分层和材料准备。因此，项目希望把分散的信息检索、资格核验、机会评估和行动规划整理为一条可追溯、可解释、可复核的智能体工作流。

（二）项目定位与参赛目标

“推免智策——大学生保研资格评估与升学规划智能体”面向大一至大三本科生，以“先核验、再评估、后规划”为设计原则。用户至少输入学生年级、学校、专业、当前专业排名和专业总人数，可进一步补充成绩单、英语成绩、竞赛科研经历、学校推免细则和培养方案。系统对资料进行结构化处理，联网核验学校推免资格，检索政策与历史信息，在能够确认的证据范围内给出阶段性机会判断。

下表把用户端最常见的痛点与系统能力逐一对应，便于说明项目不是把“搜索、评估、规划”简单并列，而是针对信息分散、数据口径混乱、低年级缺少目标和大三申请

节奏紧张等问题设置专门的处理环节。表中每一项能力均能够在后续工作流中找到对应节点，因此也是从需求到技术实现的追踪矩阵。

用户痛点	传统处理方式	本项目对应能力
政策分散且年份变化快	学生自行搜索多个官网并人工比对	联网检索+来源分级+不确定性说明
排名、名额、接收人数等口径容易混淆	依赖经验帖或单一网站结论	区分本科推荐资格与研究生接收口径
低年级不知道如何制定长期目标	仅得到“能/不能保研”的粗略回答	大一、大二输出分阶段提升计划
大三需要快速形成申请梯度	人工整理大量院校通知	符合条件后输出冲刺、匹配、相对稳妥院校建议
资料缺失或学校简称造成误判	搜索失败后直接停止	代码节点标准化+分支器控制异常流程

表 1 项目需求与功能对应关系

从信息获取的角度来看，当前保研规划面临三个核心挑战。第一，信息碎片化严重：学生需要同时关注学校教务处、本科生院、学院官网、研究生招生网等多个信息源，而这些信息源之间缺乏统一的检索入口，导致学生在规划初期就需要花费大量时间进行信息收集和整理。第二，数据口径复杂：推免名额、报名门槛、研究生接收人数、历史保研率等关键数据在不同来源中可能呈现不同的统计口径，学生若不仔细辨别，容易将研究生院接收推免生总数误认为本科推免名额，从而产生错误的概率判断。第三，规划缺乏阶段性：大一学生需要长期成长路径，大二学生需要中期目标调整，大三学生则需要短期申请执行方案，但现有工具大多只能提供一次性评估，无法根据年级特点给出差异化的规划建议。

针对上述挑战，本项目设计了一套先核验、再评估、后规划的三阶段处理流程。在核验阶段，系统首先确认学生所在学校是否真实存在且具有本科推免资格，避免后续评估建立在虚假或不适用的信息基础上。在评估阶段，系统结合联网检索获取的官方政策、学院细则和历史数据，对学生的推免机会进行多维度分析，包括排名竞争力、英语成绩达标情况、竞赛科研加分等关键因素。在规划阶段，系统根据学生所处年级生成差异化的行动方案，大一侧重长期能力建设，大二侧重中期目标调整，大三则聚焦短期申请执行。这种分阶段、分年级的设计思路，使系统能够覆盖本科全周期的升学规划需求。

在具体功能设计上，系统对每个年级的输出结构都做了明确定义。大一报告侧重基础建设，包括课程学习习惯培养、基础数学与编程能力提升、英语四级备考规划、竞赛方向认知等，帮助学生在起跑阶段就建立清晰的成长路径。大二报告侧重竞争力提升，包括高学分专业课策略、六级成绩目标、科研入门与竞赛成果积累、学院往年推免细则分析等，帮助学生将努力转化为可量化的竞争优势。大三报告侧重申请执行，包括本校资格最终确认、冲刺/匹配/相对稳妥三档院校分层、夏令营/预推免/九推时间线规划、材料准备清单等，帮助学生在有限时间内高效完成申请全流程。

在风险提示方面，系统建立了多层次的预警机制。对于大一学生，系统会提醒当前排名距离推免线还有较大差距，建议从本学期开始重点提升核心课程成绩；对于大二学生，系统会提醒英语六级成绩尚未达标，建议在大三上学期前完成六级考试；对于大三学生，当距离夏令营申请截止时间较近时，系统会提醒尽快完成目标院校筛选和材料准备。这些预警不是简单的警告标签，而是结合具体数据和时间节点给出的可操作建议，帮助学生在每个阶段都能明确下一步的行动方向。

在技术实现上，项目创新性地将大模型的语义理解能力与代码节点的确定性判断能力相结合。大模型负责理解用户输入的自然语言、总结政策文本、生成个性化报告；代码节点负责排名计算、学校名称标准化、布尔状态判断等确定性逻辑。这种模型负责理解、代码负责规则的分工模式，既保留了智能体处理复杂语义的灵活性，又确保了核心判断的可复现性和稳定性，避免了纯大模型方案中常见的结果随机性问题。

在技术实现层面，项目还特别关注了用户输入的容错性设计。考虑到学生在输入信息时可能存在表述不规范、数据格式不统一等问题，系统在资料读取节点设置了多层校验机制。例如，当用户输入“64/400”时，系统能够自动识别为排名 64、总人数 400 的格式，并进行相应的数值校验。当用户输入“哈工大”时，系统会通过联网检索确认其对应的真实学校名称，并在必要时要求用户确认具体校区。这种容错设计不仅提高了系统的易用性，也减少了因输入错误导致的后续处理失败。

二、项目阐述

（一）总体思路

项目将复杂任务拆分为“资料读取—基础校验—检索词构造—学校资格与政策联网检索—结果解析与标准化—分支判断—最终报告生成—统一输出”多个环节。需要稳定结构化判断的部分尽量交给代码和分支器；需要理解自然语言、总结政策、撰写个性化报告的部分交给大模型；需要最新公开信息的部分由联网搜索工具完成。这样避免把所有任务集中在一个模型提示词中，从结构上提高可控性。这种“代码做确定、模型做理解、工具做取证”的分工模式，是本项目工作流设计的核心原则。在实际调试中我们发现，将排名计算、学校名称标准化、布尔判断等确定性任务从大模型中剥离出来后，系统的输出稳定性和可复现性得到了显著提升——同一条输入多次运行时，关键判断结果保持一致，不再出现一次通过、一次失败的结构性波动。同时，大模型节点可以专注于政策语义理解和个性化报告生成，不再需要同时承担数值计算和逻辑判断，输出质量也相应提高。下图展示复赛版本从输入到单一结束节点的完整链路，阅读时可按“左侧输入与资料整理—中部联网检索与核验—右侧分支与统一输出”的顺序理解。



图 1 推免智策当前工作流总览

（二）输入、处理与输出边界

为了避免节点职责重叠，系统对输入、处理中间变量和最终输出进行了边界约束。表中将每个阶段可接收的数据、允许执行的动作和必须产出的结果写清楚，有助于调试时快速定位问题。输入阶段只负责接收用户的自然语言描述和附件资料，不进行任何业务逻辑判断；处理阶段包括资料标准化、学校核验、联网检索、结果解析等多个节点，每个节点只负责特定的处理任务，并通过明确的变量接口与上下游节点交互；输出阶段只负责将处理结果格式化为统一的报告结构，不重新进行业务判断。这种清晰的边界划分确保了系统的模块化和可维护性，当需要调整某一环节的逻辑时，只需要修改对应的

节点，不会影响其他环节的稳定性。同时，明确的边界约束也便于进行单元测试和集成测试，每个节点都可以独立验证其输入输出是否符合预期，从而提高整个系统的质量保障水平。

阶段	主要输入	核心处理	主要输出
输入阶段	文本、doc/docx、图片等	读取学校、专业、年级、排名、总人数及补充经历	标准化学生资料
校验阶段	标准化学生资料	缺失字段、异常排名、学校真实性和推免资格核验	有效/无效状态、学校官方全称
检索阶段	学校、专业、检索关键词	搜索学校/学院推免政策、培养方案、历史名单或名额	联网资料与来源信息
分析阶段	学生资料+核验资料	区分数据口径、分析排名位置、识别硬性条件与加分项	评估所需证据
报告阶段	综合前置结果	按年级生成不同结构的 Markdown 报告	机会评估、提升规划或院校申请指南

表 2 工作流输入—处理—输出边界

（三）设计原则

1. 证据优先：官方文件优先于非官方转载，历史名单或名额推算必须明确口径。

2. 资格与信息完整性分离：学校已确认真实且具备推免资格时，不因学院细则暂缺而错误判失败。

3. 未知不等于不满足：用户未提供英语、GPA 或挂科情况时，只记为“尚未提供”。

4. 分年级规划：大一、大二强调成长目标，大三强调资格确认和申请执行。

5. 不把模型当计算器：排名、总人数、布尔状态等稳定判断尽量由代码节点处理。

在实际工程实现中，上述原则并不是写在文档中的口号，而是直接落实到节点职责和变量含义。例如，学校真实性已经确认后，后续学院细则没有检索到，只能说明“学院级信息暂时不足”，不能倒推为“学校不存在”或“学校没有推免资格”；同样，用

户没有填写 CET-6 成绩，只能标记为“尚未提供”，不能自动判断为英语不达标。这样的边界约束能够显著减少长链路中前后结论互相矛盾的情况。

系统还特别强调“数据对象一致”。推免规划中最常见的误区，是把研究生院公布的接收推免生人数、学院允许报名的排名范围、学校本科拟推荐人数混在一起进行比较。当前工作流要求先明确数据描述的是谁、对应哪一年、属于推荐端还是接收端，再决定能否用于比例推算。只有分子和分母属于同一批本科生时，才允许进行比例换算。

对于低年级学生，系统宁可保留一定不确定性，也不会为了给出一个看似精确的数字而编造学院名额或未来政策。大一、大二报告更强调“现在的位置”和“下一阶段应该做什么”；大三报告则在核对本校资格的基础上增加院校分层、申请材料和时间线。这样的分年级设计使输出内容与学生实际行动阶段相匹配，而不是所有用户都套用同一份固定模板。

从可维护性角度看，规则型判断进入代码节点后，后续若学校政策检索策略发生变化，只需要修改检索与解析部分；若最终报告格式需要调整，也不必重写基础校验逻辑。各节点之间通过清晰变量传递，形成“修改局部、回归验证、保持其他节点稳定”的工程方式，这也是本项目在多轮调试后形成的主要实践经验。

三、产品功能

（一）学生资料读取与基础校验

系统支持用户以较自然的方式输入信息，例如“合肥工业大学、大二、64/400、软件工程”。资料读取节点负责理解用户文本和可用附件，基础资料标准化脚本进一步清洗学校、专业、年级、排名与总人数。对于“64/400”这类表达，脚本可拆分为当前排名 64、专业总人数 400，并计算排名比例。

基础资料校验采用“先合法性、后完整性”的顺序。排名和总人数是纯数值关系，必须由代码节点硬性约束；学校、专业、年级则在标准化后检查是否为空。这样可以保证异常输入在最靠前的位置被阻断，避免错误数据继续触发联网检索和长报告生成。

校验项	通过条件	失败/缺失处理
学生年级	能够识别大一/大二/大三	提示补充年级，不进入完整评估
学校	能够得到待检索学校名称	无法唯一确认时进入补充提示
专业	有明确专业名称	提示补充专业
专业排名	正整数	缺失或非正数则阻断后续
专业总人数	正整数且不小于排名	排名大于总人数时明确报错

表 3 基础资料主要校验规则

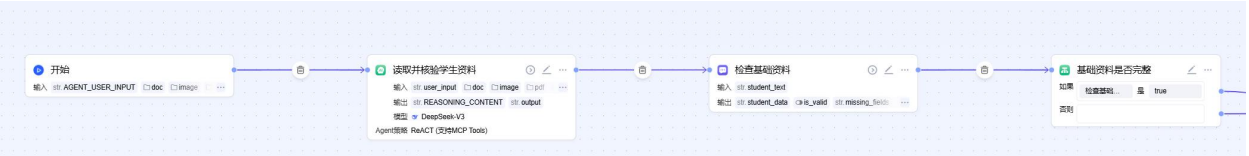


图 2 工作流局部：输入、资料读取与基础校验

该局部图重点展示用户输入进入系统后的第一段处理。资料读取节点先把自然语言转换为统一字段，随后代码节点检查排名、人数和必填项，只有通过后才允许进入检索。这种设计使“用户是否上传成绩单”与“排名信息是否有效”解耦。

（二）学校真实性与本科推免资格核验

学校核验不是简单判断搜索结果是否丰富，而是至少回答两个独立问题：第一，用户输入对应的学校是否真实存在；第二，该学校是否具有推荐优秀应届本科毕业生免试

攻读研究生资格。学校真实性和推免资格都明确通过后，school_verified 才为 True。学院归属、专业细则、专业推免率等属于后续评估信息，不能反向把已经确认的学校判为失败。

这一规则是复赛迭代中的重要稳定性优化。测试中曾出现“中国科学技术大学真实性通过、具有推免资格，但因软件工程所属学院或专业细则未完全确认而进入失败分支”的错误。当前版本通过标准化代码节点将“学校资格失败”和“后续信息不足”严格分离。

（三）政策、培养方案和历史数据检索

生成精准检索词代码节点根据学校和专业组织搜索关键词。检索目标包括学校推免办法、学院实施细则、专业培养方案、近年本科推免名单或推荐名额、竞赛科研加分规则等多个维度。代码节点会自动组合学校全称、学院名称、专业名称和年份等关键词，生成多组检索词并行搜索，确保覆盖不同的信息来源。例如，对于“合肥工业大学软件工程专业”，系统会同时生成“合肥工业大学 推免办法”“合肥工业大学 计算机学院实施细则”“软件工程 培养方案 推免”等多组检索词，从不同角度获取政策信息。这种多维度检索策略大大提高了信息获取的全面性，避免了单一关键词搜索可能遗漏的重要政策文件。

这一部分承担系统最核心的外部信息获取任务。检索词不是由最终报告模型临时自由生成，而是由代码节点从学校与专业字段构造；搜索工具返回结果后，再由普通大模型进行语义核验。这样既保留搜索覆盖面，也减少 Agent 工具协议不稳定带来的报错。



图 3 工作流局部：学校核验、检索词生成与联网资料获取

（四）机会评估与风险诊断

最终报告节点综合学生当前排名比例、已核验的学校/学院/专业参考数据、硬性条件与额外成果进行阶段性判断。报告不输出看似精确的单点概率，而采用“较高/中等/较低”等分档判断，并在证据充分时给出阶段性机会区间；所有结论都基于当前可获取

信息，不构成最终资格或录取承诺。在风险诊断方面，系统会从多个维度识别潜在风险：排名是否接近推免线边缘、英语成绩是否满足目标院校要求、竞赛科研成果是否具有竞争力、目标院校政策是否存在特殊限制。对于每一项风险，系统不仅指出问题所在，还会给出具体的改进建议和时间节点，例如“建议在大三上学期前通过英语六级”或“当前排名距推免线约 5 个名次，建议重点提升下学期 3 门核心课程成绩”。这种“诊断+处方”的模式使评估结果具有实际可操作性，而不是给出缺乏统计校准依据的单点概率。

（五）分年级升学规划

不同年级采用不同决策粒度。大一主要回答“现在打什么基础”，大二强调“如何把排名、英语、竞赛科研做成可兑现的竞争力”，大三则必须同时处理本校资格与外校申请。将年级策略固化到报告模板后，可以防止低年级过早出现院校录取承诺，也防止大三报告缺少冲刺、匹配和相对稳妥分层。

年级	报告重点	核心输出
大一	长期积累与风险预防	当前机会、排名目标、英语规划、竞赛科研启蒙、分学期计划
大二	核心课程与综合竞争力提升	排名稳定/提升目标、四六级、竞赛科研、导师和实验室准备
大三	资格确认与申请执行	本校资格评估、冲刺/匹配/相对稳妥院校、夏令营/预推免/九推时间线

表 4 分年级报告策略

分年级规划并不意味着简单地把同一组建议按年级改写。大一阶段强调课程学习习惯、基础数学与编程能力、英语四级和竞赛方向认知；大二阶段开始关注高学分专业课、六级成绩、科研或竞赛成果的形成以及学院往年推免细则；大三阶段则进入“资格确认+申请执行”的双线任务，既要守住最终排名，又要同步准备夏令营、预推免和九推材料。

在目标设置上，系统会将“保持底线、推荐目标、冲刺目标”与专业总人数结合。例如 400 人专业中，前 15%对应前 60 名，前 10%对应前 40 名。将百分比换算成名次可以

让学生更直观地理解目标，也便于在每学期成绩更新后重新评估。目标并不是官方保证线，而是依据当前排名和可核验历史信息形成的阶段性规划值。

对大三学生的院校分层，系统坚持“相对定位而非录取承诺”。冲刺院校代表当前条件有申请价值但竞争明显更强；匹配院校与本科背景、排名、英语和科研竞赛情况总体接近；相对稳妥院校只是相对于前两档风险更低。报告必须同步提示招生政策、名额和导师需求会随年份变化，最终仍需核对目标院校当年官方通知。

为了让规划具有可执行性，最终报告不会只给出“继续努力”这类泛化建议，而是尽量把任务拆成可以检查的阶段成果。例如大二阶段可以把“英语提升”进一步拆成完成 CET-6、在不影响专业课的前提下安排刷分；把“科研积累”拆成寻找导师、进入课题组、完成可展示项目、形成论文或专利等步骤。学生下一次使用智能体时，可以把新的排名、英语和成果重新输入，形成动态更新。

在输出规范方面，系统对不同类型的输出采用统一格式。所有报告包含学生基本情况摘要、学校资格确认状态、当前机会评估、排名与成绩差距分析、英语和竞赛科研建议、分阶段行动计划、信息来源与不确定性说明、免责声明等固定模块，并采用清晰的标题层级。关键数据如排名比例、推免资格状态可通过加粗等方式突出，建议和行动项使用列表呈现，便于学生定位和逐项执行。报告中的数据引用同时标注来源和年份，如“根据合肥工业大学 2025 年推免政策”，方便核实信息时效性。

在数据来源方面，系统建立了严格的可信度评估机制。所有联网检索获取的信息都会被标注来源层级：第一层为学校教务处、本科生院、学院官网等官方来源；第二层为公开的拟推荐名单、名额公示等半官方来源；第三层为网络转载、论坛讨论等非官方来源。报告中明确区分不同来源的可信度，非官方数据仅作为辅助参考，不作为主要判断依据。这种透明的数据溯源机制，使学生能够清楚了解每条信息的可靠程度。

在实际调研中，团队发现学生在保研规划过程中最常遇到的困惑集中在以下几个方面：首先，政策信息的获取成本极高，学生往往需要花费数周时间在不同官网之间反复查找，才能拼凑出相对完整的政策图景；其次，信息的时效性难以保证，很多学生参考的是往年的政策文件，而推免规则每年都会微调，过时的信息可能导致错误的决策；第三，个人情况与政策要求的匹配缺乏系统化工具，学生只能凭经验估算自己的竞争力，

无法获得基于数据的客观评估。这些问题的根源在于信息的分散性和复杂性，而非学生自身的努力程度不足。

此外，不同年级学生面临的任务差异也增加了规划的复杂性。大一学生刚刚进入大学，对推免制度了解有限，需要从基础开始建立认知框架；大二学生已经积累了一定的课程成绩和竞赛经历，需要开始有针对性地提升核心竞争力；大三学生则面临最紧迫的时间压力，需要在确认本校资格的同时完成目标院校的申请准备。现有工具大多采用一刀切的规划方式，无法根据年级特点给出差异化的建议，导致低年级学生收到过于笼统的指导，而高年级学生则缺乏足够具体的执行方案。

为了进一步提高系统的实用性，项目还考虑了多种用户场景。例如，对于没有成绩单的学生，系统支持直接输入排名和总人数；对于有成绩单的学生，系统可以解析附件中的信息并自动填充相关字段。这种灵活的输入处理方式，使系统能够适应不同学生的需求和使用习惯。

除了上述核心功能外，系统在异常处理方面也做了充分设计。当用户输入信息不完整时，系统不会直接报错，而是明确告知缺少哪些关键字段，并给出补充建议。例如，如果学生未提供专业排名信息，系统会提示请补充您的专业排名和专业总人数，例如“64/400”表示 400 人中排名第 64，而不是简单返回信息不完整。当学校名称存在歧义时，如“哈工大”可能指哈尔滨工业大学本部或威海、深圳校区，系统会列出可能的匹配项并要求用户确认。当联网检索未找到有效信息时，系统会如实说明未检索到该学校对应年份的官方推免政策，而不是编造数据。这种诚实面对不确定性的设计原则，使系统输出更加可信，也避免了因错误信息导致的决策偏差。

在输出规范方面，系统对不同类型的信息采用不同的表述方式。对于确定性事实，如学校名称、排名比例、英语成绩等，系统直接给出具体数据；对于需要判断的内容，如是否有机会获得推免资格，系统使用“根据当前信息，有机会但需要满足以下条件”等限定性表述；对于无法确认的信息，如学院级推免细则，系统明确标注尚未检索到该学院的具体实施细则，建议联系学院教务办确认。所有机会区间或定性判断均附带数据来源和适用条件，避免学生产生不切实际的期望。报告末尾统一附上免责声明，明确说明本报告仅供参考，最终资格认定以高校官方公示为准。

上述反馈机制能够帮助用户快速补齐评估所需信息，提高异常场景下的交互效率，也使系统能够在资料不完整时保持清晰、可解释的交互。

系统还会区分“获得本校推免资格”和“申请外校推免项目”两个层面。本校资格主要取决于本科院校和学院规则，而外校申请还受到本科背景、专业方向、英语、科研、竞赛、目标院校招生偏好和面试表现等因素影响。尤其在大三报告中，不能因为学生本校资格判断较为乐观，就直接推出目标院校录取判断同样乐观；两类判断必须分别表述。

对于尚未达到理想排名的学生，规划重点会从“预测结果”转向“可改变因素”。例如当前排名接近历史参考边界时，系统会优先建议提升高学分核心课程成绩、核对学院综合测评规则、选择可被学院认可的竞赛科研方向，并尽早完成英语要求。通过把注意力放到学生能够主动改变的变量上，报告的价值从一次性预测转变为阶段性行动指导。

因此，分年级规划既是产品输出的最后一层，也是前面所有核验工作的落点。只有学校资格、政策口径、学生排名和成果信息经过前置处理后，后续建议才有实际意义。该设计使“查信息”和“做规划”形成闭环，而不是把联网搜索得到的材料简单堆叠在报告中。

四、技术方案

（一）平台与节点编排

项目运行于讯飞星辰 Agent 平台，通过可视化工作流将多个能力串联。工作流并非单一对话式大模型，而是由大模型节点、代码节点、联网搜索工具节点、分支器、文本处理/输出节点共同组成。工作流中的显式连线决定数据依赖和异常路径，使每一步的输入输出均可在调试界面单独查看，便于定位“检索失败”“模型格式错误”“代码执行失败”“分支判断错误”等问题。

节点类型的选择遵循“确定性任务优先代码、模糊语义任务优先大模型、外部信息交给工具”的原则。表中不仅列出节点做什么，也解释为什么不能用另一类节点替代，从而体现工作流设计是基于工程稳定性，而不是单纯增加节点数量。

节点类型	技术职责	为什么使用该节点
大模型节点	资料理解、政策归纳、最终报告生成	适合处理非结构化文本和自然语言表达
代码节点（Python）	字段清洗、正则抽取、布尔判断、检索词拼接	稳定、可复现，避免模型输出格式漂移
联网搜索工具	获取公开网页、官方通知和历史信息	补足静态模型知识的时效性限制
分支器	依据布尔值将流程送往成功/失败路径	规则明确，避免模型自行决定流程
文本处理/统一输出	保存不同分支的最终文本并汇合	保证工作流只有一个结束出口

表 5 工作流节点类型与职责

（二）三份核心脚本

1. 生成精准检索词

脚本输入 student_data，使用正则从结构化文本或简洁用户输入中提取学校与专业，并拼接多组检索词。检索词覆盖“推荐优秀应届本科毕业生免试攻读研究生”“推免名额/拟推荐名单”“学院实施细则”“专业培养方案”等关键词。其目的不是直接判断学校政策，而是为联网搜索提供更窄、更有目标的搜索入口。

2. 学校核验结果标准化

脚本输入 verification_text，通过多组正则表达式分别识别学校真实性、推免资格、学校官方全称和明确失败证据。只有“学校真实+具有推免资格+无明确失败证据”同时成立时，school_verified 才为 True。脚本特别规定：学院归属、专业细则、专业推免率缺失不参与学校资格失败判断。

3. 基础资料标准化

脚本负责抽取学校、专业、年级、专业排名、专业总人数，兼容“64/400”等简写格式，并检查排名、总人数是否为正数以及排名是否超过总人数。脚本返回 data_valid、error、rank_ratio 等字段，供后续分支器与报告节点使用。

三份脚本分别位于资料标准化、检索前处理和学校核验后处理三个关键位置。它们均只依赖 Python 标准库，输入和输出字段保持简单，以便在星辰平台代码节点中直接复现。脚本不负责生成长文本，而是为大模型和分支器提供更可靠的中间变量。

脚本	关键输入	关键输出	主要依赖
01_生成精准检索词.py	student_data	school_name、major_name、output	Python 标准库 re
02_学校核验结果标准化.py	verification_text	verification、school_verified、school_name	Python 标准库 re
03_基础资料标准化.py	student_data	student_data、school_name、major_name、grade、rank、total、rank_ratio、data_valid、error	Python 标准库 re

表 6 三份 Python 脚本接口摘要

（三）学校核验算法逻辑

学校核验标准化节点采用“正向证据+明确失败证据”的组合判断。正向证据包括“学校真实性核验：通过”“学校推免资格：具有/具备”等表达；失败证据包括“学校真实性核验：未通过”“不具备推免资格”等明确否定。这样能够兼容上游大模型不同

但语义一致的表述，同时防止“未找到专业推免率”之类无关信息触发错误失败。在算法实现上，脚本通过多组正则表达式分别匹配正向证据和失败证据，并设置严格的优先级规则：只有学校真实性和推免资格均有明确正向证据、且不存在明确失败证据时，school_verified 才设为 True；出现明确失败证据，或现有信息不足以确认学校真实性时，school_verified 设为 False 并进入对应提示。对于“无法确认”和“明确不具备推免资格”两类情况，输出文本分别说明原因，避免把信息不足表述成资格不具备。此外，脚本还会提取学校官方全称，确保学校名称在整个工作流中保持一致。图中展示学校核验结果从自然语言转化为 Boolean 后如何进入分支器。关键设计是 school_verified 只代表“学校真实且具有本科推免资格”，学院归属、专业推免率和具体细则缺失不会把 Boolean 改成 False，从流程层面解决真实高校被误判的问题。



图 4 工作流局部：结果标准化、学校资格布尔判断与分支

（四）联网检索策略与数据可信度

联网资料使用分层策略。第一层优先学校教务处、本科生院、学院官网、研究生院/研究生招生网等官方来源；第二层使用公开的拟推荐名单、名额公示或本科生规模进行历史推算；第三层才参考非官方转载或经验帖，且必须标注来源层级。这种分层策略确保了政策信息的可信度和准确性，官方来源的信息直接使用，第二层信息需要经过推算和验证，第三层信息仅作为参考且必须明确标注。在实际检索过程中，系统会自动识别搜索结果的来源域名，对于 edu.cn 等官方教育域名的结果给予更高的权重和优先级，对于论坛、博客等非官方来源的结果则降低权重并在报告中明确标注信息来源。同时，系统还会对检索到的政策文件进行年份识别，优先使用最新年份的政策文件，对于过时的政策信息会标注“历史信息，仅供参考”，避免学生因使用过时政策而做出错误决策。

推免率是整个项目最容易产生误判的数据。该表将可以直接使用、可以条件推算以及禁止直接使用的口径分开列出。尤其需要区分“本科拟推荐人数”和“研究生接收推免生人数”，也不能把“前 30%可报名”直接理解成“推免率 30%”。

数据类型	可否直接用于本科推免覆盖判断	处理原则
学校/学院官方推免名额	可以	优先采用，注意专项计划和适用届次
本科拟推荐名单人数	可以（需对应本科生规模）	可用于历史比例推算
“排名前 X%可报名”	不可以直接当推免率	只能视为申请/资格门槛
研究生接收推免生人数	不可以	属于研究生招生接收口径
非官方保研率统计	可辅助	需多来源交叉并标注非官方

表 7 推免数据口径处理规则

（五）大模型节点约束与输出稳定性

大模型节点使用明确的系统提示词约束报告结构、数据口径和不确定性处理。例如：未知信息不得直接视为不满足；学校已核验通过后不得因学院信息缺失重新判失败；大三必须保留冲刺、匹配、相对稳妥三档院校建议。最终报告节点设置更高的最大输出 Token，以避免较长的大三报告在“信息来源”或“院校推荐”章节中途截断。

在平台调试中还发现，Agent ReACT 类型节点可能出现推理格式协议错误，因此当前关键解析与报告生成更倾向使用普通大模型节点配合前置工具节点，降低工具调用协议对内容生成的干扰。联网工具负责搜索，大模型负责读取搜索结果，而不是让同一个 Agent 同时完成搜索决策与长文本生成。

（六）统一输出与单结束节点

工作流采用多个业务分支、一个最终结束节点的结构。成功评估、学校核验失败、资料缺失等分支分别生成对应文本，并统一写入 final_reply，再由读取节点送入唯一结束节点。这样既保持各分支相互独立，又使前端只需维护一个输出字段；当输出异常时，也可以直接检查对应分支写入的变量值进行追溯和定位，从而降低多分支维护与调试成本。

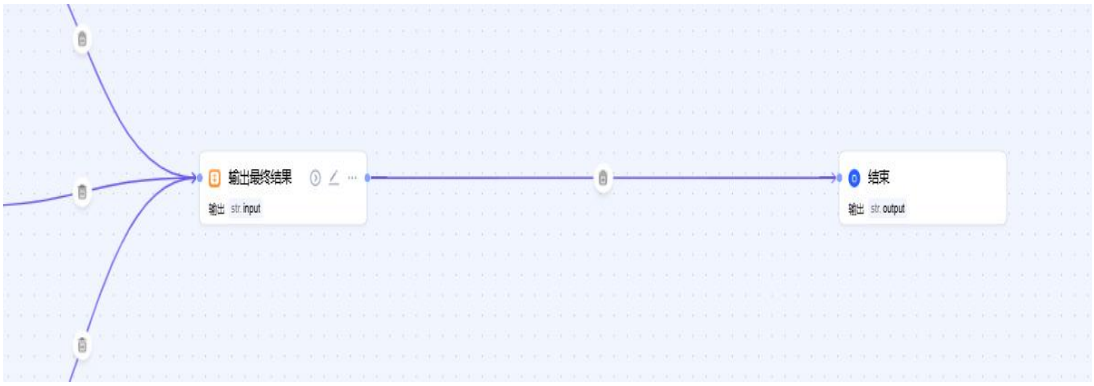


图 5 工作流局部：报告生成、失败提示与统一输出

（七）异常处理与调试机制

下表来自真实调试过程，记录错误现象、根因和最终工程处理方式。它反映出本项目的迭代不是只调整提示词，而是同时处理模型协议、搜索参数、布尔逻辑和输出 Token 等问题。

异常处理既覆盖业务异常，也覆盖平台异常。业务异常包括资料缺失、数值错误和学校无法核验；平台异常包括 Agent 推理协议错误、内容审核、搜索结果过长和模型输出截断。项目把每次真实故障转化成回归用例，形成持续迭代机制。

异常类型	典型现象	当前处理
基础资料缺失	缺学校/专业/排名/总人数	基础校验阻断并提示缺失字段
数值异常	排名大于总人数	代码节点返回 data_valid=False
学校简称/歧义	HFUT、哈工大等	可唯一确认则标准化；不能唯一确认则提示补充
联网接口异常	第三方 API 错误、URI 过长	缩短检索输入、控制 full_text、减少单次请求体积
Agent 推理格式错误	缺少 ReACT 标识字段	关键节点改用普通大模型+前置联网工具
内容审核中断	网页全文带入无关内容	关闭 full_text 或缩小上下文，只传与推免相关摘要
长报告截断	输出在章节中途结束	提高最终报告节点 maxTokens 并控制重复内容

表 8 已处理的典型异常与工程措施

异常处理不仅覆盖明显的缺字段和数值错误，也覆盖“搜索成功但结论不一致”“模型输出过长被截断”“网页正文导致上下文异常”等工程问题。项目将每一次真实调试故障记录为可复现样例，并在修复后加入回归测试。这样做能够避免同类问题在后续版本中重新出现，也使测试报告不仅展示成功案例，还能体现工作流对异常条件的处理能力。

在调试顺序上，团队遵循“先看失败节点，再看节点输入，再看输出格式，最后调整提示词”的原则。例如聚合搜索节点绿色成功而后续解析模型失败时，不应反复修改搜索词；学校核验文本已经明确写出“真实性通过、具有推免资格”而分支仍走失败路径时，应优先检查 Boolean 标准化代码。通过缩小修改范围，可以减少一次改动引发多个新问题的概率。

此外，系统将最终报告生成与联网检索拆开后，输出稳定性得到明显提升。最终报告节点只读取已经整理的 student_data 和 verification 等结果，不再自行调用工具，避免在生成长报告时同时承担网络搜索协议。对于大三长报告，提升最大输出 Token 后，能够完整保留冲刺、匹配、相对稳妥院校以及申请行动计划，避免内容在中途被平台硬截断。

从工程视角看，异常处理还承担“保护最终报告”的作用。只要前置数据中存在明显冲突，例如排名大于总人数、学校真实性无法确认、学校明确不具有推免资格，流程就不应继续生成长篇机会评估和规划。相反，如果只是学院实施细则尚未检索到，则应继续保留学校层面的有效信息，并把缺失内容放入最终报告的不确定性部分。这样的分级处理可以避免“一处信息缺失导致全流程失败”。

调试过程中还建立了输入输出对照记录。每个关键节点至少保留一组正常样例和一组异常样例，用于检查变量名称、类型和内容是否符合下游预期。例如生成精准检索词节点必须稳定输出 school_name、major_name 和 output；学校核验标准化节点必须输出 Boolean 类型的 school_verified。只要数据契约稳定，后续模型和分支器的维护成本就会明显降低。

在平台实际运行中，大模型服务、搜索工具和代码节点的失败表现不同，因此也采用不同恢复策略。搜索工具超时应优先检查检索词长度和工具参数；模型内容过长应检

查上下文和 maxTokens；代码节点失败则直接检查参数名、返回字段和 Python 语法。把故障按节点类型分类后，团队能够更快完成定位，减少无关节点被反复修改。

整体而言，技术方案的重点不在于堆叠尽可能多的节点，而在于让每个节点只做最适合自己的任务。当前版本已经形成比较稳定的职责分工：大模型理解和表达、代码确定性校验、工具联网取证、分支器路由、变量节点统一结果。后续功能扩展也将尽量沿用这一结构，避免重新回到单节点承担过多职责的状态。这种职责清晰的架构设计带来了三个方面的工程优势：第一，问题可定位——当输出出现异常时，可以通过检查各节点的输入输出快速判断问题出在哪个环节，而不需要把整个工作流当成黑盒；第二，修改可隔离——调整某一节点的逻辑不会影响其他节点的稳定性，例如修改检索词生成脚本不会影响学校核验算法和最终报告格式；第三，能力可扩展——当需要增加新功能时，可以在现有架构中插入新的节点，而不需要重构整个工作流。在复赛迭代过程中，团队深刻体会到，智能体的工程稳定性往往不取决于模型有多强大，而取决于架构设计是否合理、节点职责是否清晰、异常处理是否完善。本项目的技术方案正是在这一理念指导下逐步完善的，每一次节点拆分和职责调整都带来了可观测的稳定性提升。

五、运行效果

（一）正常场景

以“合肥工业大学、大二、64/400、软件工程”为例，系统能够识别学校、年级、专业、当前排名和总人数，计算排名比例为前16%。随后系统自动核验合肥工业大学是否具有推免资格，并联网检索该校计算机学院的推免实施细则和软件工程专业的培养方案。在正常场景下，系统会完整输出学生基本情况摘要、学校推免资格核验结果、当前保研机会评估、排名差距分析、可提升方向建议、分阶段行动计划和信息来源说明。测试结果表明，系统能够正确走通“资料读取→基础校验→学校核验→联网检索→结果解析→分支判断→最终报告”的完整流程，各节点输入输出符合预期，报告结构完整、数据口径一致。

（二）大三场景

大三用户除了本校推免资格判断外，更关注外校申请。当前报告结构要求在具备评估基础后完整输出“冲刺院校2所、匹配院校2所、相对稳妥院校2所”，每所院校均需说明推荐理由、竞争强度、申请时间节点和材料准备要点。测试中以“合肥工业大学、大三、64/400、软件工程”为例，系统能够正确识别学生处于推免边缘区间，并据此给出差异化的院校分层建议：冲刺院校选择学科实力更强但竞争激烈的985高校，匹配院校选择与本科背景和排名相当的211高校，相对稳妥院校选择与当前条件更匹配、相对申请风险较低的同层次高校。同时，报告还会给出夏令营、预推免和正式推免三个阶段的时间线规划，以及个人陈述、推荐信、成绩单等材料的准备清单。测试结果表明，大三场景下的报告结构完整、院校分层合理、建议具有可操作性，能够有效帮助大三学生制定系统的申请策略。

（三）异常与边界场景

运行效果不只看“有没有文字输出”，还要检查是否走对分支、是否使用正确的数据层级、报告结构是否与年级一致。表中场景作为日常回归测试的基础用例，覆盖了正常输入、资料缺失、学校核验失败、排名异常等多种情况。对于每一种异常场景，系统

都应该有明确的处理逻辑和输出格式，而不是简单地报错或返回空结果。例如，当用户输入的学校无法核验时，系统应该明确告知“未找到该校的官方推免信息”，并建议用户确认学校名称或提供更多信息；当用户缺少排名或总人数时，系统应该列出缺失的关键字段，并给出补充建议。通过这种规范化的异常处理机制，系统能够在各种输入条件下保持稳定的输出质量，避免因异常输入导致的崩溃或错误判断。

边界测试关注“系统什么时候必须拒绝继续”。例如排名 420/400 属于确定性错误，应在代码节点直接阻断；学校简称若能唯一确认则自动标准化，存在歧义时要求用户补充；英语成绩缺失只记为“尚未提供”，不直接判断不满足。这些边界条件的设计体现了“确定性错误硬阻断、不确定性信息软处理”的原则。对于排名超过总人数、排名为 0 或负数等明显的输入错误，系统会在代码节点直接拦截，并给出明确的错误提示，避免这些无效输入进入后续的大模型节点造成资源浪费和结果混乱。而对于学校名称简称、英语成绩缺失等不确定性信息，系统则采用更加灵活的处理方式：能够自动标准化的就自动处理，无法确定的就标注为“待确认”并在报告中说明，而不是简单地拒绝服务。这种差异化的边界处理策略既保证了系统的稳定性，又最大化了服务的可用性。

测试场景	预期系统行为
缺少专业	提示补充专业，不生成单点概率
缺少排名	提示缺少排名
排名 420/400	提示排名不能大于总人数
HFUT	若检索唯一对应合肥工业大学，则标准化后继续
学校真实但学院细则缺失	学校资格仍通过，缺失项进入不确定性说明
专业无法核验	提示专业/培养方案信息不足，不编造专业政策

表 9 典型运行场景与预期效果

（四）输出规范

最终结果统一采用中文 Markdown 结构。大一、大二报告一般包含学生基本情况、学校推免资格核验、当前保研机会评估、排名与政策门槛差距、额外加分分析、风险提示、提升方案、分阶段计划、信息来源与不确定性说明等模块；大三报告在此基础上增

加院校分层推荐、申请时间线和材料清单。所有报告均要求明确标注信息来源和年份，对于无法确认的信息必须标注“待确认”或“不确定”，不得直接视为不满足条件。在表述方式上，系统坚持“数据+解释+建议”的三段式结构：先给出具体的数据和判断结果，再解释数据背后的含义和政策依据，最后给出可操作的建议和时间节点。这种统一的输出规范确保了不同年级、不同场景下的报告具有一致的结构和质量标准，便于学生快速定位所需信息，也便于后续的功能扩展和质量监控。

（五）补充数据集的实测结果与回归价值

本轮补充数据集来自工作流多次真实调用记录，包含正常报告、模型协议错误、内容审核提示、报告截断以及学校核验分支误判等情况。与人工编写的理想样例相比，这类“带故障的数据”更能检验工作流是否具有工程鲁棒性，因此本次报告将其作为运行效果和回归测试的重要补充。

以合肥工业大学为例，数据集中同时存在大一 56/400 并带 B 类赛事奖项、大二 64/400 无额外成果、大三 64/400 等多种组合。它们可以分别检验额外加分信息是否被保留、排名边界是否被正确理解以及大三是否能够进入院校分层路径。数据记录还出现过“学校真实性核验：通过、学校推免资格：具有，但最终进入学校核验失败”的矛盾输出，该问题正是学校核验布尔值重新定义后的重点回归项。

数据集样例	主要观察	当前回归要求
合肥工业大学·大一·56/400·含 B 类赛事奖项	能够读取排名前 14%及竞赛经历；早期曾出现 Agent 协议/内容审核问题	保留成果信息；资料读取节点不联网；后续按大一结构输出
合肥工业大学·大二·64/400·软件工程	多次调用曾得到不同单点概率，暴露推免率口径与学校数据层级的重要性	优先使用学校/学院真实数据；禁止固定单点概率映射
合肥工业大学·大三·64/400·软件工程	曾出现“学校通过却走失败分支”以及长报告截断	school_verified 必须为 True；大三必须完整输出 2 冲+2 匹配+2 相对稳妥
清华大学/中国科学技术大学同类输入	用于检验高推免环境下不同学校不能被统一套用同一比例	学校级、学院级和专业级数据分层使用；未知项写入不确定性

表 10 补充数据集中的代表性回归样例

补充数据还记录了 Agent ReACT 推理格式错误、full_text 导致上下文风险、maxTokens 过低导致长报告中途停止等现象。这些记录说明，智能体的评测不能只关注内

容是否“看起来合理”，还必须关注执行链是否稳定、分支是否正确和长文本是否完整。当前版本把这些历史失败场景全部转化为固定测试资产。

从数据集可以看到，最终输出的机会分档或阶段性区间只是面向用户的阶段性结论，真正需要稳定的是其背后的证据链和逻辑方向。例如前 16%在不同学校中的含义不同，系统必须根据各校实际推免环境重新解释；同样，学院细则暂时缺失只能降低精度，不能推翻已经确认的学校推免资格。

六、迭代计划

（一）近期迭代：稳定性与覆盖度

1. 扩大回归测试集，覆盖不同排名区间、缺失字段、学校简称、专业无法核验等输入。

2. 继续校准学校核验标准化脚本，减少“学校已通过但被失败分支截断”的误判。

3. 优化搜索词长度与网页摘要输入，降低URI过长、模型内容审核和上下文膨胀问题。

4. 对长报告设置输出长度监控，避免大三多院校推荐在中途截断。

近期迭代的核心目标是“把当前工作流做稳”，优先解决可复现性、数据口径和边界条件问题，而不是盲目增加功能。团队认为，一个稳定可靠的基础工作流比功能丰富但经常出错的系统更有价值。因此，近期迭代将重点放在回归测试集的扩充、学校核验脚本的校准、搜索词优化和长报告完整性监控四个方面。每一项迭代都有明确的验收标准，需要通过固定测试集的回归验证后才能合并到主版本。

短期迭代的具体实施计划包括以下几个方面：第一，在学校名称标准化方面，建立一个包含常见简称和官方全称的映射数据库，覆盖985、211、双一流等主要高校，减少因名称不匹配导致的核验失败。第二，在学院级数据检索方面，优先补充热门学院的推免细则和历史数据，如计算机学院、软件学院、电子信息学院等，提高信息的针对性和实用性。第三，在测试覆盖方面，增加更多边界条件测试，如排名刚好在推免线边缘、学校当年政策有重大调整、学生跨专业申请等场景。第四，在用户体验方面，优化报告的可读性，增加更多可视化元素，如排名趋势图、目标院校对比表等。

上述四项工作均来自实际调试中发现的真实问题，每完成一项都需要通过固定测试集的回归验证，确保稳定性提升不会引入新的分支错误。测试集覆盖度的扩大和学校核验脚本的校准是基础工作，搜索词优化和长报告监控则直接影响用户体验的完整性，四者并行推进，共同提升智能体在复杂输入下的工程鲁棒性。

（二）中期迭代：政策索引与历史对比

计划维护学校和学院推免政策的更新时间、来源地址和适用届次，使智能体能够提示“当前引用的是哪一届政策、学生实际推免年份是否可能发生变化”。同时可增加历史评估对比功能，记录学生排名、英语、竞赛科研等指标的变化，用趋势而非一次快照支持规划。政策索引功能将按“学校—学院—专业—届次”四层结构组织，每条政策记录文件名称、发布时间、适用对象、关键规则摘要和官方来源链接。当学生输入年级后，系统自动匹配对应届次的政策，并在报告中标注政策的有效年份和可能的更新风险。历史评估对比功能则允许学生保存每次评估结果，系统自动生成排名变化曲线、英语成绩进度和竞赛科研积累时间线，帮助学生直观看到自己的成长轨迹和与目标的差距，从而更有针对性地安排下一阶段的提升任务。

（三）长期迭代：申请执行闭环

长期方向是把院校匹配进一步连接到申请执行：目标院校清单、材料缺口、夏令营/预推免时间、个人陈述与简历准备、面试复习任务等均可形成可跟踪的计划。项目仍坚持“辅助决策”定位，不替代高校正式资格审核。在申请执行闭环中，系统将根据学生的目标院校清单自动生成时间线提醒，包括夏令营报名截止、预推免材料提交、九推系统开放等关键节点。材料缺口检测功能会对比目标院校的申请要求和学生已准备的材料，自动列出缺失项并给出准备建议。个人陈述和简历准备模块可提供结构模板和内容提示，帮助学生有条理地组织自己的经历和优势。面试复习任务则根据目标专业的常见面试方向生成复习清单。整个闭环始终以“辅助学生更高效地完成申请”为目标，所有正式资格审核和录取决定仍由高校官方负责，系统不做任何录取承诺。

七、应用价值

（一）对学生的价值

项目把“找到正确政策”“看懂政策口径”“代入自己的排名与经历”“知道下一步做什么”四个环节连接起来。低年级学生可以更早建立保研目标和风险意识，避免到大四才发现自己不符合基本条件；高年级学生则能够在有限时间内高效完成信息搜集和院校筛选，把精力集中在提升核心竞争力和准备申请材料上。与传统的“学长学姐经验帖”或“论坛零散信息”相比，本系统的核心优势在于信息的时效性和口径的一致性。所有政策信息均来自官方渠道并标注年份，避免了学生因使用过时政策而做出错误决策。同时，系统会根据学生的具体情况进行个性化分析，而不是给出通用的“保研攻略”。例如，对于排名在推免线边缘的学生，系统会重点分析可提升的课程和竞赛机会；对于排名靠前的学生，系统会更多关注目标院校的选择和申请策略的优化。这种个性化的规划建议能够帮助每一位学生找到最适合自己的保研路径。

（二）对高校升学指导的价值

对于辅导员、学生组织或学院升学指导场景，大量常见问题具有高度重复性，例如“排名到多少有希望”“某类竞赛是否加分”“英语是否有门槛”。通过维护官方来源和规则化工作流，可把通用咨询转化为结构化初步答复，再由人工处理个别复杂情况。学院可以将系统嵌入现有的升学指导公众号或网站，作为学生自助查询的第一入口，减少辅导员在重复性问题上的时间消耗。辅导员和升学指导老师则可以通过后台查看学生常见问题的统计分布，了解哪些政策最容易被误解、哪些学校的信息最常被检索，从而有针对性地组织讲座和编写指导材料。对于批量学生的情况，系统还可以生成学院整体的推免环境分析报告，包括本届学生的排名分布、竞赛科研参与度等，为学院制定升学指导策略提供数据参考，使升学指导工作从“被动答疑”转向“主动规划”。

（三）对同类智能体的参考价值

项目展示了一种适用于高校政策类任务的通用架构：联网检索解决时效性，大模型解决非结构化理解，代码节点解决稳定字段和规则，分支器解决流程控制，最终报告负

责解释性输出。该方法可以迁移到奖学金资格、转专业、竞赛加分认定、交换项目申请等场景。这类场景的共同特点是：政策分散在多个官方来源、数据口径容易混淆、需要结合个人情况进行判断、结论需要可解释和可追溯。本项目的“先核验、再评估、后规划”流程、学校资格与学院信息分离的 Boolean 设计、推免率口径区分规则等，都可以直接复用到其他政策类智能体的开发中。此外，项目在调试过程中积累的异常处理经验——如 Agent 推理协议错误、搜索结果过长触发上下文问题、长报告输出截断等——也为同类智能体的工程化提供了参考，帮助开发者避免重复踩坑，更快地从原型搭建过渡到稳定可用的产品状态。

综合来看，本项目的应用价值体现在“学生个体—高校组织—技术生态”三个层面。对学生而言，它降低了信息获取门槛，提供了可执行的规划路径；对高校而言，它分流了重复性咨询，提升了升学指导的整体效率；对同类智能体开发而言，它提供了一套经过真实调试验证的政策类任务架构和异常处理经验。三个层面的价值相互支撑，使项目不仅是一个可用的工具，也具有一定的方法论参考意义。后续随着政策数据的持续积累和用户规模的扩大，项目的应用价值还将进一步提升，特别是在数据驱动的升学指导和个性化规划方面，有望形成更具深度的服务能力。

八、商业模式

（一）基础服务模式

面向学生提供基础的学校资格核验、政策检索、阶段性评估和行动规划，可作为开放或免费基础功能，重点建立准确性和用户信任。基础服务的核心价值在于降低学生获取保研信息的门槛，让每一位学生都能够快速了解自己的推免资格和当前定位。通过免费提供高质量的基础评估服务，项目可以积累大量的用户反馈和使用数据，为后续的功能优化和政策数据维护提供支持。同时，基础服务也是建立品牌认知和用户信任的重要途径，当学生认可了基础评估的准确性和实用性后，更有可能愿意为增值服务付费。在基础服务阶段，团队将重点关注评估结果的准确性、政策信息的时效性和用户体验的流畅性，确保每一位用户都能够获得可靠、有用的保研规划建议。

（二）增值服务模式

在政策数据维护和评测体系成熟后，可围绕多校对比、目标院校收藏、申请时间线、材料清单、历史评估对比等提供增值功能。所有增值功能仍应保留来源链接和不确定性提示，避免将评估结果包装成确定性承诺。增值服务的核心价值在于为学生提供更深入、更个性化的保研规划支持。例如，多校对比功能可以让学生同时查看多所目标院校的推免政策、录取难度和申请时间，便于做出更明智的选择；目标院校收藏功能可以帮助学生跟踪关注院校的政策更新和截止日期，避免错过重要时间节点；历史评估对比功能可以记录学生每学期的排名、成绩和成果变化，生成趋势分析，帮助学生直观地看到自己的进步和需要改进的方向。这些增值功能均建立在基础评估的准确性之上，通过持续的政策数据维护和用户反馈优化，不断提升服务质量和用户价值。

（三）B 端合作可能

可与学院、学生工作部门或升学指导组织合作，按学校/学院维护官方政策索引和常见问题库，为学生提供统一入口。B 端价值主要来自信息维护效率、重复咨询分流和统一口径，而不是替代教师或学院进行正式资格认定。

商业化设计坚持“基础评估低门槛、增值功能围绕持续规划”的方向。核心价值不是售卖一个固定单点概率，而是把政策更新、多校对比、材料管理和长期成长记录变成持续服务。

模式	主要用户	核心价值	注意事项
基础学生端	大一至大三本科生	资格核验、规划建议	明确辅助决策属性
增值学生端	进入申请季的学生	多校对比、时间线、材料管理	不承诺录取结果
学院/组织端	学院、辅导员、学生组织	统一政策入口、常见咨询分流	需要持续维护官方数据

表 11 商业与推广模式概览

从落地路径看，项目当前阶段仍以“提升学生信息获取和规划效率”为首要目标，不将机会判断包装成收费承诺。基础版本可以提供学校资格核验、政策检索和分年级规划；更高阶功能可围绕多校对比、申请材料清单、时间节点管理和个人成长档案展开。即使未来形成增值服务，也必须保持所有资格与录取结果由高校官方决定的边界。在产品迭代节奏上，建议先把基础功能的准确性和稳定性做到行业领先水平，通过用户口碑建立品牌信任，再逐步推出增值功能。这种“先免费做深、再增值做宽”的路径，既符合教育类工具的用户获取规律，也能避免过早商业化对产品口碑的损害。基础功能的持续优化本身就是最好的用户获取渠道——当学生发现系统给出的资格判断和政策解读比自己手工检索更准确、更高效时，自然会形成自发传播和用户粘性。

对于学院或学生组织，B 端合作可围绕政策索引维护、学生自助查询入口和升学指导数据支持展开。合作可按学期或年度进行：团队负责系统功能维护、工作流更新与政策信息结构化，学院负责确认本校官方政策口径和更新节点。系统可嵌入现有公众号或网站，为学生提供统一查询入口，并向学院提供常见问题统计和匿名化趋势分析。涉及正式资格认定、学生个人信息和政策解释的内容仍由学院官方确认，系统仅提供辅助服务。

商业模式能否成立的关键不只是模型效果，还包括数据更新能力和结果可信度。推免政策每年变化，如果不能持续维护来源和年份，历史信息很快就会失去价值。因此后续若扩展服务，应把“政策更新时间、适用届次、来源链接和学院归属”作为基础数据资产，而不是只保存模型生成的结论。数据资产的建设需要持续投入，但它也是项目最

核心的竞争壁垒——当其他同类产品还在依赖通用大模型临时搜索时，本项目已经积累了按学校—学院—专业—届次组织的结构化政策索引，检索效率和准确性都会显著更高。此外，结果可信度还体现在透明度上：每一条结论都应能追溯到具体的官方来源和年份，用户可以自行核对。这种“可追溯的可信度”是教育类工具的生命线，也是商业模式能够长期成立的基础。一旦出现政策引用错误或年份混淆，用户信任会迅速崩塌，因此数据质量管控必须放在与产品功能开发同等重要的位置。

九、团队介绍

（一）团队信息

队伍名称为 Postgraduate Recommendation Assistant，学校为合肥工业大学宣城校区。团队成员均为本科生，专业方向与软件工程相关，具备扎实的编程基础和系统开发能力。在项目开展过程中，团队成员充分发挥各自的专业特长，形成了高效的协作模式。工作流开发成员负责讯飞星辰 Agent 平台的节点编排、Python 脚本编写和模型提示词调优；测试成员负责设计测试用例、复现异常场景和执行回归测试，确保系统的稳定性和准确性；文档成员负责整理技术方案、撰写应用报告和准备答辩材料，将复杂的工程逻辑转化为评审专家易于理解的表达。团队成员之间保持密切沟通，每周定期召开项目进展会议，及时解决开发过程中遇到的技术难题和协作问题。

团队采用开发、测试和文档并行协作的工作模式。工作流开发负责节点与脚本的实现和调试，功能测试负责复现异常和执行回归验证，报告成员负责把工程逻辑转化为评审可理解的材料，从而形成闭环。这种并行协作模式大大提高了项目的推进效率：开发成员在实现新功能的同时，测试成员可以对已有功能进行回归测试，文档成员可以同步整理技术文档和应用方案。三者之间通过共享的项目管理工具和版本控制系统保持信息同步，确保任何一方的变更都能及时被其他成员感知和响应。

成员	专业	主要职责
徐培智	软件工程	项目负责人、智能体工作流开发
步凡	软件工程	报告撰写
韩林默	软件工程	功能测试
林臻阳	软件工程	功能测试
肖翔	软件工程	报告撰写

表 12 团队成员与分工

（二）迭代协作机制

开发阶段由工作流负责人完成节点编排、提示词、代码逻辑和平台调试；测试成员使用不同学校、年级、排名和异常输入进行回归测试，记录失败节点和错误表现；报告

成员同步整理项目背景、技术路线、运行效果和测试结论。发现问题后先定位属于“模型理解、联网工具、代码节点还是分支逻辑”，再做局部修改，尽量避免大范围改动引入新的回归问题。

团队协作以工作流版本为核心进行同步。开发成员负责节点连接、变量设计、代码规则和模型参数；测试成员围绕正常、缺失、异常、简称和大三长报告等场景建立固定测试用例；报告成员把每轮技术变化同步记录到应用方案和 README。这样可以保证演示材料、代码、测试说明和实际线上版本保持一致，避免“文档写的是旧流程、平台跑的是新流程”。

每一次关键修改都要求至少完成三类回归：一是正常样例能否继续生成完整报告；二是历史故障样例是否确实不再复现；三是本次修改是否引入新的分支错误。例如修改学校核验脚本后，需要同时测试真实有资格高校、真实但无推免资格高校和虚构高校；调整长报告 Token 后，还要确认大一、大二输出不会因为长度增加而出现不必要重复。

十、技术深化说明

（一）工作流分层设计

复赛版本采用分层工作流而不是单一大模型直接回答：

1. 输入层负责承接用户文本和附件；
2. 理解层负责把自然语言整理为学生资料；
3. 规则层使用 Python 处理排名、学校名称、布尔状态等确定性信息；
4. 检索层使用聚合搜索获取公开政策与历史数据；
5. 控制层通过分支器选择正常评估、资料缺失或学校核验失败路径；
6. 输出层最终生成分年级报告。

分层的主要价值是“问题可定位”。当用户得到错误结果时，可以判断问题究竟来自字段读取、检索结果、核验脚本、分支条件还是最终报告提示词，而不是把整个智能体当成不可解释的黑盒。

分层表进一步说明每一层发生错误时会影响哪一段结果。通过把问题局部化，可以在调试日志中迅速确定应该检查模型、代码、搜索还是分支条件。

层级	主要节点	核心职责	输出
输入层	开始	接收文本、DOC/PDF/图片/表格	原始输入
理解层	资料读取大模型	提取年级、学校、专业、排名、成果	student_data
规则层	3 个 Python 代码节点	字段清洗、检索词、学校核验	结构化字段/Boolean
检索层	聚合搜索	获取政策、培养方案、历史数据	search_data
控制层	分支器+变量节点	控制成功/失败路径	final_reply
输出层	最终报告大模型	生成大一/大二/大三差异化报告	Markdown 报告

表 13 工作流分层设计与各层职责

从工程角度看，分层并不是简单增加节点数量，而是降低每个节点的职责复杂度。输入、理解、规则、检索、控制和输出六层分别对应不同失败模式；一旦某层出现问题，可以在该层独立复现和修复，而不需要重写整个工作流。

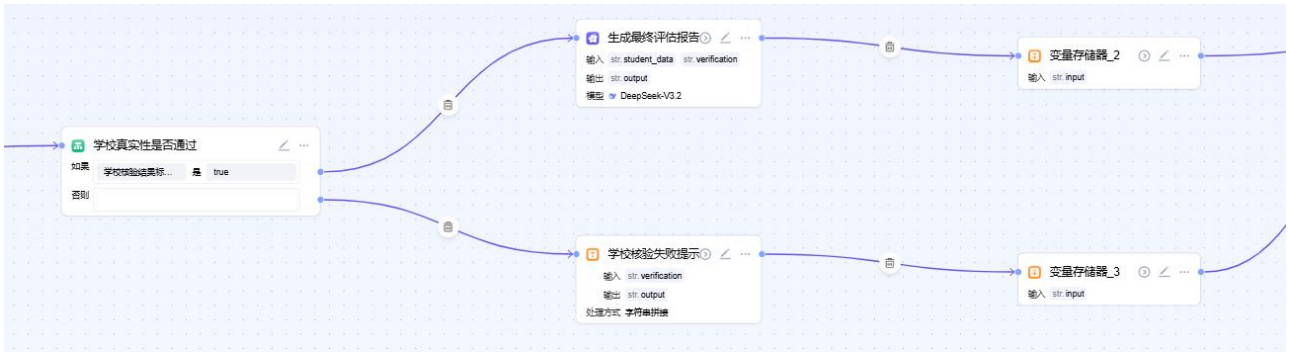


图 6 工作流前段：输入、资料读取与基础校验

（二）节点间变量与数据契约

为了减少节点之间的歧义，工作流使用稳定变量传递核心数据。大模型输出可以是较长自然语言，但进入程序判断前需要通过代码节点转化成稳定字段。Boolean 变量只承担“是否通过”的含义，长文本负责解释，不把多个概念混到一个字段里。这种设计的核心思想是“让确定性的东西确定化，让模糊性的东西集中化”。排名、人数、资格状态等可以用简单类型表达的信息，全部通过代码节点固化为数值或布尔值，下游分支器和报告节点直接读取这些稳定字段，不需要再次从自然语言中解析。而政策摘要、风险解释、规划建议等需要语义理解的内容，则保留为长文本字段，由大模型节点负责生成和消费。通过这种分离，工作流中每个节点的输入输出都有明确的类型预期，节点之间的协作不再依赖模糊的自然语言理解，稳定性得到根本性提升。

变量契约是保证多节点协同的基础。字段名称、类型和含义一旦稳定，下游节点就不需要反复从长文本里猜测信息；尤其 Boolean 变量必须保持单一语义，避免逻辑污染。在实际调试中，我们曾遇到过因变量含义不清晰导致的分支错误：某个布尔字段同时承载了“学校真实性通过”和“学院信息完整”两个含义，导致学院信息缺失时被误判为学校不真实。修复方式是将学校资格判断拆为 school_real（学校真实性）、push_qualified（是否具有推免资格）和 explicit_failure（是否存在明确失败证据）三个独立判断维度，再由三者综合得出 school_verified 结果，确保每个 Boolean 只承载单一语义。其中，school_real、push_qualified 和 explicit_failure 仅作为学校核验脚本内部的中间判断变量，不作为跨节点字段传递；节点对外统一输出 school_verified，因此表 14 仅列出工作流节点间实际传递的核心变量。这次经验使团队深刻认识到，变量契约不是文档中的形式化要求，而是直接影响工作流正确性的核心

设计。每个变量在定义时都必须回答三个问题：它的类型是什么？它的取值范围有哪些？它在什么条件下会被修改？只有这三个问题都有明确答案，这个变量才能进入正式的数据流。

变量命名和类型决定了节点之间是否真正“可组合”。student_data 承载语义资料，school_verified 只承载布尔判断，verification 承载可供报告引用的证据文本。明确数据契约后，分支器无需重新理解自然语言，稳定性明显提高。在命名规范上，团队遵循“名词+下划线+限定词”的模式，例如 school_verified 表示学校资格是否通过，data_valid 表示基础资料是否有效，rank_ratio 表示排名比例。这种命名方式使变量含义在代码中一目了然，不需要额外注释就能理解。在类型选择上，能用布尔值就不用字符串，能用数值就不用文本，因为简单类型的比较和判断更不容易出错。例如排名比例用浮点数存储，而不是用“前 16%”这样的字符串，下游节点可以直接进行数值比较和区间判断。这些看似细节的设计选择，在复杂工作流中累积起来，会显著降低出错概率和调试成本。

变量名	类型	来源	下游用途
student_data	String	资料读取/基础标准化	检索和最终报告主输入
school_name	String	精准检索词/学校核验	统一学校名称
major_name	String	精准检索词	限定专业检索范围
search_data	Object/String	聚合搜索	推免资料解析
verification	String	学校核验标准化	最终报告证据
school_verified	Boolean	学校核验标准化	学校资格分支
final_reply	String	各结果存储节点	唯一结束节点输出

表 14 节点间核心变量与数据契约

数据契约特别强调 school_verified 的含义：它只回答“学校是否真实且具有本科推免资格”。学院归属未确认、专业推免率缺失、学院实施细则暂未检索到，都属于后续评估信息不足，不能把 school_verified 改成 False。这一约束是复赛迭代中最重要的稳定性修复之一。前述中国科学技术大学误判问题的根因，就是 school_verified 的语义被污染，混入了学院级和专业级的信息完整性判断。修复后，school_verified 严格限定在学校层面，学院和专业信息的完整性通过单独的字段或在 verification 文本中标记为“尚未确认”来表达。这种“层面分离”的设计原则也推广到了其他变量：每个变量只回答一个层面的问题，不跨层混合判断。例如 rank_ratio 只表达学生在本专业的排名

位置，不掺杂对推免资格的判断；english_status 只表达英语成绩是否提供，不判断是否达到目标院校要求。通过这种严格的语义隔离，工作流中每个判断都有明确的依据和范围，不会出现“一个字段出错导致整条链路误判”的连锁反应。

（三）大模型与代码节点职责边界

项目经过多轮调试后形成明确的职责边界：大模型处理非结构化理解和自然语言生成，代码节点处理确定性的规则判断。模型擅长理解用户不同表达方式、总结政策文本、组织规划建议；代码更适合判断排名是否合法、字段是否存在以及学校核验布尔值。

这一对比表明确了哪些任务需要语义理解，哪些任务应交给程序。通过职责边界，模型可以集中处理政策语义和规划生成，代码则提供可复现的数值与状态判断。

项目将“模糊语义”和“确定规则”分开处理。竞赛经历、政策摘要和规划建议适合大模型；排名大小关系、字段存在性和布尔判断适合代码。将二者结合，可以同时获得灵活性与可复现性。

任务	大模型适合度	代码适合度	当前实现
识别“64/400”为排名和人数	高	中	资料读取+代码兜底
判断 64≤400	中	高	代码
总结学院推免细则	高	低	普通大模型
判断 school_verified	中	高	代码
生成完整规划报告	高	低	大模型
生成稳定检索词	中	高	代码

表 15 大模型与代码节点职责边界

这种设计保留了智能体的核心智能能力，同时避免把可以精确程序化的逻辑交给概率性模型。这种职责划分在实际调试中得到了充分验证。例如，在学校核验环节，如果将学校是否真实存在的判断交给大模型，可能会因为模型对某些学校名称的不熟悉而产生误判；而通过代码节点进行精确匹配，则可以确保判断的一致性和准确性。同样，在排名计算环节，如果让大模型直接计算排名比例，可能会因为浮点数精度或表达方式的差异导致结果不一致；而通过代码节点进行数学计算，则可以保证每次输出完全相同。这种确定性任务下沉的策略，是系统稳定性的核心保障。

在变量契约方面，项目建立了严格的命名规范和类型约束。所有 Boolean 变量均使用语义明确的命名，如 school_verified 表示学校资格是否通过、data_valid 表示基础资料是否有效，使变量含义在代码中一目了然。所有数值变量均使用浮点数存储，避免字符串比较带来的歧义。所有文本变量均明确标注来源和可信度，如 policy_source 字段区分官方文件、学院通知、网络转载等不同来源。这种契约式设计使节点之间的协作不再依赖模糊的自然语言理解，而是基于明确的类型预期，从根本上提升了工作流的稳定性，从而提升复现性。

（四）联网检索稳定性设计

早期版本曾尝试让 ReACT Agent 在节点内部自主调用搜索工具，但实际调试出现过“无效插件名称”“无效推理格式”“缺少必要标识字段”等协议类错误。当前版本将联网动作独立为聚合搜索工具节点，再将工具结果传给普通大模型解析。

这一改造将“工具是否调用成功”和“模型是否能够理解搜索结果”拆成两件事：搜索失败时检查工具输入；模型解析失败时检查上下文和提示词。问题定位更加直接。

联网方案经历了从 Agent 内部自主调用到独立工具节点的调整。对比结果说明，工具调用和结果解析拆开后，更容易确认请求是否成功，也减少模型输出工具协议格式错误。

设计项	早期方案	当前方案	收益
联网调用	ReACT 内部工具调用	独立聚合搜索	减少协议错误
检索词	模型自由生成	Python 代码构造	稳定、可复现
结果解析	与工具调用耦合	普通大模型单独解析	便于调试
上下文	full_text 可能过长	控制摘要/结构化结果	降低截断和审核风险

表 16 联网检索方案迭代对比

联网模块从 ReACT Agent 内部工具调用改为独立搜索节点，是一次关键架构调整。工具是否成功、搜索结果是否相关、模型是否能解析被拆成三个可观察步骤，既减少协议错误，也方便展示完整的技术证据链。

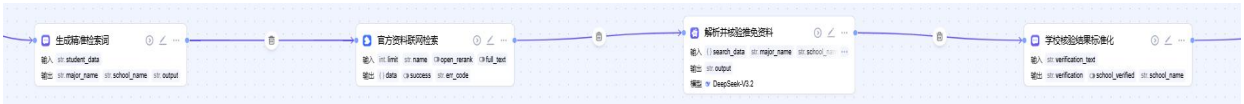


图 7 工作流中段：精准检索与联网资料解析

十一、节点级实现说明

（一）开始与资料读取节点

开始节点接收用户文本以及可选的 DOC、PDF、图片和表格附件。资料读取节点的任务仅限“读取和整理”，不在这一节点联网搜索，也不计算最终机会分档或区间。这样可以避免输入理解与外部工具调用相互干扰。

用户不必强制上传成绩单。如果文本已经写明“合肥工业大学、大二、64/400、软件工程”，系统应直接识别年级、学校、排名、人数和专业。只有当关键字段确实缺失时才进入补充提示。

资料读取节点必须兼容自然语言、模板字段和附件补充三种输入方式。其目标是最大限度利用用户已经提供的信息，同时坚持“未提供就标记未提供”，不自行猜测学院或个人经历。

用户输入	识别结果	处理
合肥工业大学、大二、64/400、软件工程	五项核心字段齐全	进入后续校验
合肥工业大学、大二、软件工程	缺排名与总人数	提示补充
大一，中科大，软件工程，56/400	可识别简称但需后续标准化	继续检索官方全称
附件为空	无附件信息	直接跳过附件读取

表 17 资料读取节点的输入识别策略



图 8 输入到基础资料校验的实际工作流局部

（二）基础资料标准化脚本

基础资料标准化脚本承担第一个确定性检查。脚本提取学校、专业、年级、排名和总人数，兼容“64/400”写法，检查排名和总人数必须为正值且排名不能大于总人数，同时计算 rank_ratio=rank/total×100%。

这些异常规则全部属于确定性逻辑，因此放在 Python 脚本中执行。这样同一条 420/400 输入无论模型状态如何变化，都必须得到“排名不能大于总人数”的一致结果。

基础标准化脚本把自然语言中容易变化的表达转化成稳定字段，并对排名关系进行数学检查。这里使用代码而不是模型，是因为“排名不能大于总人数”不存在语义歧义，程序判断更快也更可靠。

检查项	合法条件	异常例子	动作
年级	大一/大二/大三	空	提示缺失
学校	可检索的学校名	空	提示缺失
专业	明确专业名	空	提示缺失
排名	rank>0	0 或负数	停止
总人数	total>0	0	停止
关系	rank≤total	420/400	停止

表 18 基础资料标准化脚本的异常处理规则

脚本输出 data_valid 后，分支器不再需要理解长文本，只需要读取 Boolean。这种“模型负责提取、代码负责校验”的组合比完全由模型判断稳定。

（三）精准检索词生成脚本

精准检索词脚本从 student_data 中提取 school_name 与 major_name，再围绕“本科推荐免试资格、学校/学院实施细则、培养方案、拟推荐名单、推荐名额和保研率”构造搜索词。其目标是减少过宽搜索带来的噪声，同时保留官方政策和历史比例两类信息。

检索词采用多方向覆盖，而不是只查“推免政策”。学校资格、学院细则、培养方案、历史名单和保研率辅助信息各有独立作用，后续解析节点再根据来源和口径决定采用程度。精准检索词脚本的作用是缩小搜索空间。它围绕学校官方推免办法、学院实施细则、培养方案和历史名单分别构造关键词，使后续模型接收到的不是泛化搜索结果，而是与当前学生更相关的资料。

三份脚本的设计遵循最小依赖、最大稳定原则。选择 Python 标准库 re 而非第三方库，是因为在线代码节点的运行环境对第三方包的支持有限，使用标准库可以避免安装失败或版本冲突问题。脚本的输入输出字段名称与工作流节点保持严格一致，例如学校核验脚本接收 verification_text 字段，输出 verification、school_verified 和 school_name 三个字段，确保数据在节点间流转时不会因为字段名不匹配而丢失。这种字段契约式的设计，使脚本可以独立测试，也可以在工作流中稳定运行。

在联网检索方面，系统采用了多方向覆盖、结果合并的策略。精准检索词脚本会同时构造多个搜索方向：学校官方推免办法、学院实施细则、培养方案、历史拟推荐名单、推免率统计数据等。每个方向都有独立的搜索词组合，确保不同来源的信息都能被检索到。搜索结果进入解析节点后，系统会根据来源权威性和数据口径进行筛选和合并，优先采用官方文件中的数据，网络转载的信息作为补充参考。这种多源采集、权威优先的策略，既保证了信息的覆盖面，又控制了数据的可信度。

检索方向	示例结构	目的
学校资格	学校+推荐优秀应届本科毕业生免试	确认学校是否有推免资格
学院政策	学校+专业+推免实施细则	查专业所属学院具体规则
培养方案	学校+专业+本科培养方案	查核心课程与学分要求
历史数据	学校+专业+拟推荐名单+名额	寻找真实推荐人数
比例辅助	学校+保研率+推免率	官方未公布比例时交叉参考

表 19 精准检索词脚本的主要检索方向

检索脚本本身不决定数据真假，只负责把问题问得更精准。数据可靠性由后续普通大模型结合来源、年份和数据口径判断。

（四）推免资料解析与数据口径控制

联网结果进入普通大模型后，解析节点需要形成统一结构：学校真实性、学校推免资格、官方全称、学院归属、学校政策、学院政策、近年本科推免数据、参考推免比例、额外加分规则、硬性资格条件、官方来源、非官方辅助来源以及尚未确认的信息。

这一节点最重要的不是“找一个数字”，而是识别数字口径。比如“研究生院接收推免生 1000 人”可能包含大量外校学生，不能直接当作该校本科推免名额；“专业排名前 30%可以申请”只是资格门槛，也不能直接等同于 30%的保研率。

表中的例子用于训练和测试解析节点的“数据口径意识”。当分子分母无法对应时，系统宁可保留不确定性，也不进行二次假设；这一原则直接影响最终机会判断是否可信。

资料解析节点重点判断“这条数据到底说明什么”。官方报名门槛、推荐名额、拟推荐名单和研究生接收人数对应不同业务含义，若不先区分口径，再精确的数字也可能导致错误的机会判断。

原始信息	是否可直接作为本科推免率	说明
官方明确公布本科推免比例 20%	是	口径明确
学院拟推荐 80 人/本科 400 人	可推算	分子分母一致
前 30%可申请	否	申请门槛
接收推免研究生 1000 人	否	研究生招生口径
第三方平台称 55%	辅助	需多来源交叉验证

表 20 本科推免数据口径辨析示例

（五）学校核验结果标准化脚本

学校核验标准化脚本是实际调试中最关键的修复点之一。此前出现过“中国科学技术大学学校真实性核验通过、具有推免资格，但系统仍输出学校信息核验未通过”的矛盾。根因是旧逻辑把学院未知、专业比例未知等后续问题混入学校资格布尔值。

当前脚本将判断拆为 school_real、push_qualified 和 explicit_failure。只有学校真实且具备本科推免资格，并且没有明确失败证据时，school_verified=True。学院、专业细则、专业推免率、加分分值是否完整，都不影响这个学校层面的 Boolean。

学校核验标准化把模型自然语言结论转换成 school_verified，并提取学校官方全称。其核心原则是学校层面判断与学院层面信息解耦，确保真实且具有推免资格的高校不会因专业数据暂缺而被误判。

核验结果	school_verified	解释
真实+有推免资格+学院未知	True	学院未知不影响学校资格
真实+有推免资格+专业比例未知	True	比例未知进入不确定性说明
真实+明确无推免资格	False	学校层面失败
无法确认学校真实性	False	评估对象无法确认

表 21 学校核验布尔值的典型判定场景

（六）分支、变量存储与唯一结束节点

学校核验代码输出 school_verified 后，分支器只判断该 Boolean 是否为 True。True 进入最终评估报告，False 进入学校核验失败提示。资料缺失路径则在更前面被基础资料分支拦截。

不同路径最终都将文本保存到统一变量 final_reply，再由读取节点送入唯一结束节点。这样前端只需要消费一个输出字段，工作流改动时也不需要维护多个结束节点。

三类路径对应三类用户体验：缺信息就告诉用户补什么，学校失败就解释为什么停止，学校通过则生成完整报告。最终全部转成 final_reply 输出，使前端调用保持一致。

路径	触发条件	主要输出
基础资料缺失	核心字段缺少或数值异常	补充信息提示
学校核验失败	基础资料校验通过后，school_verified=False	学校信息核验失败说明
正常评估	school_verified=True	完整推免机会与规划报告

表 22 三类流程路径与统一输出方式

分支节点本身只读取 Boolean，不再执行复杂语义判断。正常报告、学校失败提示和资料缺失提示分别生成后统一保存，最终通过单一出口返回，降低多路径维护成本。

这种分支设计的核心优势在于故障隔离。当系统输出异常时，可以通过检查 final_reply 的内容快速判断问题出在哪个分支：

如果输出是资料缺失提示，说明问题出在基础资料标准化脚本；

如果输出是学校核验失败提示，说明问题出在学校核验环节；

如果输出是完整报告，则需要进一步检查报告内容是否符合预期。

这种通过输出反推故障节点的调试方法，比逐节点检查效率高出数倍，也使系统在生产环境中的运维变得更加可行。

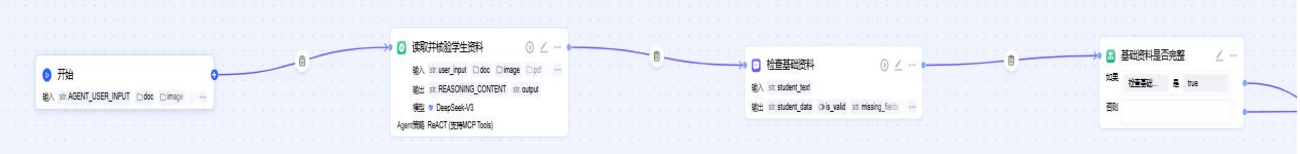


图 9 后半段分支、结果存储与统一输出

唯一结束节点的设计还带来了前端对接的便利。由于所有路径最终都输出到同一个字段 final_reply，前端只需关心这一个输出接口，不需要为不同路径设计不同的展示逻辑。这种后端多路径、前端单出口的架构，降低了前后端的耦合度，也使系统在后续功能扩展时更加灵活。例如，如果未来需要增加新的异常路径（如学校已停招、专业已合并等），只需在后端增加新的分支和提示词模板，前端无需任何改动即可支持。

（七）最终评估报告节点

最终报告节点是普通大模型节点，不再承担联网工具调用。它接收 student_data 与 verification 等前置结果，按照系统提示词中定义的固定结构生成最终评估报告。选择普通大模型节点而非 Agent ReACT 节点的原因是，最终报告生成阶段不需要工具调用，普通节点的输出更稳定，不会出现推理格式协议错误。该节点的系统提示词经过多轮调优，明确规定了报告的章节结构、数据表述方式、不确定性处理原则和免责声明格式。对于不同年级的学生，系统提示词会引导模型采用不同的报告侧重点：大一、大二强调成长规划和能力提升，大三强调资格确认和申请执行。通过这种“前置证据+固定结构+分年级策略”的设计，最终报告节点能够稳定输出结构完整、内容详实、口径一致的评估报告。

大一、大二输出学生基本情况、学校资格、当前机会、排名差距、额外加分、风险、提升方案、分阶段计划、信息来源与不确定性；大三必须进一步包含冲刺院校 2 所、匹配院校 2 所、相对稳妥院校 2 所，以及夏令营、预推免、九推阶段的行动计划。

最终报告结构不是一套模板简单替换年级，而是三个阶段目标不同的模板。大三的 6 所院校分层是强制输出项，同时必须声明“相对稳妥”并不代表保证录取。

最终报告节点只消费前置已经核验的资料，不再自行联网或重新判断学校真实性。通过这种“前置证据—最终表达”模式，可以减少模型在最后一步推翻前面已确认事实的情况。

年级	报告重点	额外要求
大一	长期成长、基础课程、英语、竞赛科研启蒙	不做过早录取承诺
大二	排名提升、核心课、六级、竞赛科研积累	给保持/推荐/冲刺目标
大三	本校资格+申请执行	必须保留 2 冲+2 匹配+2 相对稳妥

表 23 最终评估报告的分年级结构

（八）长报告参数与输出完整性

最终报告结构较长，尤其大三需要同时输出本校资格分析、三档 6 所院校建议和申请时间线。实际测试中，maxTokens=2048 时经常出现报告在“分阶段规划”或“信息来源”中间突然结束，并且每次截断位置不同。

该现象属于输出长度上限，而不是模型主动认为报告已完成。当前版本已将 maxTokens 设置为 8192；如果平台允许且大三报告仍然很长，可继续提高。temperature 和 topK 保持相对稳定即可。

长报告节点的参数直接决定输出完整性。多次调用记录中出现过报告在“信息来源”或“分阶段规划”中途停止，最终确认与 maxTokens 过低有关，因此将输出预算提升是结构性修复。

长报告完整性还受到模型输出 Token 限制影响。实际测试中 maxTokens=2048 会在不同章节随机截断，因此复赛版本将输出预算提高，并用大三长报告作为压力测试，确保信息来源和免责声明也能够完整输出。在参数调优过程中，团队还发现了一个容易被忽视的问题：输出 Token 预算不仅影响报告的完整性，还会影响模型的生成质量。当剩余 Token 不足时，模型可能会出现“仓促结尾”的现象——在报告的最后几个章节中，建议变得简略、分析变得肤浅，甚至出现逻辑跳跃。这种现象在 maxTokens=2048 时尤为明显，大三报告的“申请行动计划”章节经常只有一两句话就草草结束。将 maxTokens 提升到 8192 后，模型有足够的空间展开每个章节的内容，报告的整体质量和完整性都得到了显著改善。此外，团队还在最终报告节点的系统提示词中增加了“输出完整性检查”指令，要求模型在生成报告前先评估内容长度，如果预计可能超出 Token 限制，则优先保证核心章节（资格判断、院校分层、行动计划）的完整性，次要章节可以适当精简。这种“核心优先”的策略进一步降低了关键信息被截断的风险。表 24 总结了长报告模型参数的调优建议，包括 maxTokens、temperature、topK 等关键参数的推荐取值和调整依据。

参数	问题表现	当前设置/建议
maxTokens	2048 时长报告随机截断	当前版本 8192；必要时继续提高
temperature	过高会导致表达漂移	0.5 附近保持稳定
topK	过大增加候选发散	当前 4 可保持
searchDisable	最终节点无需再次联网	依赖前置 verification

表 24 长报告模型参数调优建议

十二、测试与质量保障

（一）测试设计原则

测试集不仅验证“能输出”，更关注分支正确性、数据口径和历史问题回归。复赛版本的测试数据主要使用合肥工业大学、清华大学、中国科学技术大学三所学校，覆盖普通完整输入、信息缺失、数值异常、学校简称、学院细则缺失和大三院校分层等场景。

测试维度同时覆盖业务逻辑和工程稳定性。年级、学校名称、缺失字段、排名边界、学校核验和长报告结构都必须出现，才能避免只测“正常样例”导致隐藏问题遗漏。

测试采用“正常主链路+边界条件+历史故障回归”三类组合。仅测试顺利输入无法证明工作流稳定，真正容易暴露逻辑缺陷的是简称、缺失字段、异常排名和学校资格通过但学院信息不完整等场景。

维度	覆盖内容	目的
年级	大一/大二/大三	验证报告结构分流
学校名	全称/简称/歧义	验证标准化和失败路径
资料完整性	完整/缺专业/缺排名	验证前置拦截
排名边界	64/400、420/400	验证数值规则
学校核验	通过但学院未知	验证不误判
大三输出	冲/匹配/稳	验证最终结构完整

表 25 回归测试覆盖维度

（二）十组核心测试用例

核心测试用例以合肥工业大学、清华大学和中国科学技术大学为主，便于在相同输入格式下比较不同学校的推免环境和分支行为，也方便每次修改后重复执行。

十组核心用例尽量复用合肥工业大学、清华大学和中国科学技术大学，减少学校差异带来的额外变量，同时覆盖年级、简称、缺失信息和数值异常。测试数据中的个人排名和成果只用于功能验证，不对应真实个人。

编号	学校	输入特征	主要预期
T01	合肥工业大学	大一 64/400	正常大一评估
T02	中国科学技术大学	大二 64/200	正常大二评估
T03	合肥工业大学	大三 32/400+竞赛+英语成绩	输出三档院校
T04	合肥工业大学	HFUT 简称	标准化官方全称
T05	合肥工业大学	合工大简称	标准化官方全称
T06	清华大学	缺专业	提示补充
T07	合肥工业大学	缺排名	提示补充
T08	中国科学技术大学	420/400	阻断后续
T09	虚构大学 A	学校不存在	核验失败
T10	测试高校 B	模拟：无推免资格	提示无资格

表 26 十组核心测试用例

这些用例可以在每次版本修改后重复执行。只要某个历史问题再次出现，就说明新修改破坏了已有逻辑，需要回滚或进一步定位。

在回归测试执行方面，团队建立了标准化的测试流程。每次版本修改后，必须按照以下顺序执行测试：首先运行正常样例，确认基本功能没有退化；然后运行历史故障样例，确认已修复的问题没有复发；最后运行边界条件样例，确认新修改没有引入新的异常。测试结果需要记录在案，包括每个样例的执行时间、关键节点的输出状态、最终报告的完整性等信息。如果某个样例从通过变为失败，需要立即定位问题原因并进行修复，直到所有样例都恢复通过状态。

在测试数据管理方面，团队对测试用例进行了系统化分类和编号。每个测试用例都包含以下字段：用例编号、输入数据、预期输出、实际输出、执行状态、发现日期、修复日期、关联问题编号。这种结构化的测试数据管理方式，使团队能够快速追溯某个问题的历史修复记录，也便于在评审阶段展示系统的稳定性。测试数据本身也接受质量检查，确保样例标题、输入内容和预期输出之间的一致性，避免因为测试数据本身的错误导致误判。

（三）历史问题与回归测试

历史问题表将开发中的失败案例保留下来作为回归资产。一个问题被修复后并不删除，而是转化为固定测试项，防止后续修改重新引入同类错误。

历史故障被保留为长期回归样例。例如学校核验误判、不同学校推免率趋同、ReACT格式错误、网页全文触发内容审核、长报告被 Token 截断等，都要求在新版本修改后再次测试，防止旧问题复发。

历史问题	表现	根因	修复
学校资格误判	中科大/川大学校通过却走失败分支	布尔值混入学院信息缺失	重写学校核验代码
保研率趋同	不同学校单点概率异常接近	检索过窄/兜底逻辑固定	增强真实数据口径
ReACT 错误	缺少推理标识/无效插件名称	Agent 工具协议不稳定	改为独立搜索+普通大模型
内容审核	模型收到网页全文后报违规倾向	full_text 引入无关正文	控制搜索返回内容
报告截断	写到一半结束	maxTokens=2048	提高输出 Token

表 27 历史问题、根因与修复方式

回归测试遵循“最小化改动范围”原则：一次只修改一个问题相关节点，并保留已经正常工作的节点。这样可以确保每次修改的影响范围可控，便于定位问题和验证修复效果。例如学校核验脚本对某些学校名称的匹配存在偏差时，只修改核验脚本中的匹配规则，不同时调整上游检索词生成脚本或下游最终报告提示词，避免多个改动叠加后难以判断真正起作用的修复项。

在测试覆盖方面，项目建立了正常场景+边界场景+异常场景三层测试体系。正常场景验证系统在标准输入下的输出质量，包括报告完整性、格式规范性和内容准确性。边界场景验证系统在临界条件下的判断能力，如排名刚好在推免线边缘、学校刚好具有推免资格等。异常场景验证系统在错误输入下的容错能力，如虚构学校、排名大于总人数、关键信息缺失等。三层测试相互补充，确保系统在各种情况下都能给出合理响应，而不是只在理想条件下才能正常工作。这样既降低改动风险，也便于确认修复是否真正有效。

回归测试的核心价值在于防止问题复发。每次修复一个问题后，对应测试用例都会被永久保留并纳入后续版本；当新功能开发或参数调整时，直接重复固定回归集即可快速判断是否破坏已有能力，使版本迭代保持可控。

（四）补充数据集的故障样例转化

补充数据集中的异常记录被划分为“模型协议类、联网上下文类、逻辑分支类、输出长度类”四类。每一类都对应明确的复现步骤和验收标准，避免问题修复后失去测试依据。

模型协议类以 ReACT 节点“无效的推理格式、缺少必要标识字段”为代表，验收标准是使用独立搜索工具加普通大模型后不再出现该协议错误；联网上下文类以 full_text 输入过长触发内容审核为代表，验收标准是搜索结果经过控制后能够进入解析节点；逻辑分支类以真实高校误入失败分支为代表，验收标准是 school_verified 语义保持单一；输出长度类以长报告随机截断为代表，验收标准是大三报告能够完整结束于免责声明。

这些故障样例使测试从“输入—输出对照”升级为“问题—根因—修复—回归”闭环。对于竞赛评审而言，它也能说明当前工作流并非一次搭建完成，而是经过多轮真实运行后逐步稳定。

本次复赛回归统一采用表 26 所示的十组核心测试用例（编号 T01 至 T10）。最新数据集的价值在于把“正常业务场景”和“故障边界场景”放在同一套固定输入中。每次修改脚本、提示词、模型或搜索参数后，都可以直接重复这 10 组输入并记录结果。若某一用例从通过变成失败，就能够快速判断新版本存在回归。测试数据中的排名、英语和竞赛经历均为功能验证数据，不对应真实个人，因此也适合在演示和评审阶段进行脱敏展示。

为了让测试结果可比较，建议对每组用例记录至少四项内容：工作流是否完整执行、关键 Boolean 是否正确、最终输出是否符合年级结构、是否出现未提供信息被错误推断。对于大三样例，还要额外检查冲刺、匹配、相对稳妥三档是否全部存在，以及是否出现“稳录”“保证录取”等不恰当承诺。通过固定验收项，可以把主观的“看起来没问题”转化为较稳定的检查标准。

测试集中的学校名称也用于验证不同类型的学校核验逻辑。合肥工业大学用于常规推免资格和简称场景；清华大学用于高推免环境下的长期规划、缺字段和数值异常测试；中国科学技术大学用于学校资格与学院信息分离、专业细则不足等场景；无加分用例则验证系统不会虚构成果。不同类型输入组合能够覆盖工作流最重要的路由边界。

除功能正确性外，回归还关注输出稳定性。相同输入多次运行时，学校真实性、核心字段和分支路径应保持一致；最终自然语言可以存在表达差异，但不能出现一次判断学校有资格、另一次又判断无资格的结构性波动。如果发生这类波动，应优先把相关判断进一步下沉到代码或固定规则。

在评审演示中，测试数据还承担可复现展示作用。团队可以选取一组正常大二、一组大三有竞赛成果、一组排名异常和一组虚构高校，分别展示正常报告、分年级差异、前置代码拦截和学校真实性失败。这样能够在有限演示时间内体现工作流不仅会生成报告，还真正具备数据校验和异常分支能力。

为了让测试集不仅承担“演示输入”的作用，复赛版本进一步把每一组样例与中间变量对应起来。正常样例除了检查最终报告是否生成，还要确认 data_valid、school_verified 等关键状态是否符合预期；缺失字段样例则检查工作流是否在正确节点停止，而不是在后续大模型中才用自然语言补救。这样可以把测试结果从“最终文字看起来合理”提升为“流程状态也正确”，更适合工程化验收。

补充数据集中的学校简称、排名异常和缺失字段样例具有不同测试目标。HFUT 简称用于验证能否通过联网结果标准化为官方全称；420/400 用于验证确定性的数值错误是否被代码节点直接拦截；缺专业、缺排名用于验证前置拦截，无加分用例验证系统不虚构成果。三类样例分别对应名称标准化、数值校验和完整性拦截，不能使用同一套兜底逻辑处理。

测试数据本身也需要接受一致性检查。例如样例标题、用户实际输入和预期结果应指向同一所学校、同一种故障类型；如果测试描述与输入内容不一致，回归结论就可能失真。因此团队在后续版本中会把“测试数据质量”纳入测试流程：先检查样例字段，再执行工作流，最后同时记录节点状态、关键检索结果和最终报告。这样既能提高复现效率，也能为评审现场的功能演示准备更稳定的样例组合。

十三、部署复现与依赖说明

（一）完整平台依赖

三份 Python 脚本只是工作流中的确定性规则部分，不能单独运行出完整智能体。完整复现依赖讯飞星辰 Agent 平台的工作流能力、大模型服务、聚合搜索工具、代码节点、分支器、变量/文本处理和结束节点。

依赖项	作用	是否必须
讯飞星辰 Agent 平台	节点编排、变量传递、调试运行	是
大模型节点	资料读取、政策解析、最终报告	是
聚合搜索工具	获取互联网公开资料	是
代码节点	执行三份 Python 脚本	是
分支器	控制成功/失败路径	是
变量/文本节点	统一 final_reply	是

表 28 完整智能体的平台依赖

README 中需要把“Python 依赖”和“平台依赖”分开写清楚，避免评审误以为只运行.py 文件即可复现整个产品。

在平台依赖方面，系统需要以下核心组件协同工作：第一，大模型服务负责自然语言理解和生成，推荐使用具备长上下文能力的模型以支持大三长报告的完整输出；第二，联网搜索工具负责获取最新的官方政策和历史数据，需要支持多源聚合搜索以提高信息覆盖度；第三，代码节点负责确定性规则判断，包括排名计算、学校名称标准化、布尔状态控制等；第四，分支器负责流程控制，根据前置节点的输出选择不同的处理路径；第五，变量存储节点负责跨节点数据传递，确保信息在工作流中完整流转。这些组件缺一不可，任何单一组件的缺失都会导致系统无法正常运行。

在部署建议方面，项目推荐按照先本地验证、再平台部署的顺序进行。本地验证阶段可以使用 Python 脚本单独测试代码节点的逻辑正确性，确保排名计算、学校名称标准化等功能在各种输入下都能给出预期结果。平台部署阶段需要在讯飞星辰 Agent 平台上建立完整的工作流结构，配置各节点之间的变量引用关系，并进行端到端的功能验证。建议在部署前先使用 10 组核心测试用例进行回归测试，确保所有场景都能通过后再上

线。上线后还需要定期检查联网搜索工具的可用性和大模型服务的稳定性，及时处理可能出现的异常情况。

在平台部署方面，系统提供了详细的配置说明。工作流的每个节点都有明确的输入输出变量定义，便于在不同环境中复现。代码节点的 Python 脚本使用标准库实现，无需额外安装第三方包，降低了部署门槛。联网搜索工具的配置包括搜索源优先级、结果数量限制和超时设置等参数，可以根据实际需求进行调整。这种模块化的部署方案，使系统能够灵活适应不同的使用场景。

（二）Python 与库依赖

复赛版本的三份代码节点均使用 Python 3.x 标准库实现，主要依赖 re 正则表达式模块，无需额外安装第三方 pip 包。采用最小依赖设计有利于在线代码节点快速复现，也能减少不同运行环境之间的版本差异。

脚本	标准库	第三方库	核心作用
01_生成精准检索词.py	re	无	提取学校/专业并构造搜索词
02_学校核验结果标准化.py	re	无	判断学校真实性和推免资格
03_基础资料标准化.py	re	无	字段校验、排名比例计算

表 29 三份 Python 脚本的库依赖

如果未来独立部署为 Web 系统，可再增加 HTTP 客户端、数据库、文档解析等依赖；但当前复赛交付尽量保持最小依赖，以提高可复现性。

复现时应先在讯飞星辰 Agent 平台建立与报告一致的节点结构，再逐一配置变量引用。三份 Python 脚本虽然不依赖第三方 pip 包，但其输入字段名称必须与工作流节点一致，例如学校核验脚本接收 verification_text，输出 verification、school_verified 和 school_name 等字段。在实际复现过程中，建议按照以下步骤进行：第一步，创建工作流并添加所有节点，包括开始节点、资料读取大模型节点、三个代码节点（基础资料标准化、精准检索词生成、学校核验结果标准化）、聚合搜索工具节点、解析大模型节点、分支器、最终报告大模型节点、变量存储节点和结束节点；第二步，按照报告中的节点连接关系配置连线，确保数据流向正确；第三步，逐一配置每个节点的参数，包括大模型节点的系统提示词、代码节点的 Python 脚本、工具节点的搜索参数、分支器的判

断条件；第四步，配置变量引用，确保每个节点的输入字段正确引用上游节点的输出；第五步，使用测试用例进行验证，按照“基础资料缺失样例—正常大二样例—学校核验异常样例—大三长报告样例”的顺序逐步测试，前一类通过后再验证下一类，可以快速定位变量引用、代码返回结构、搜索工具权限或模型输出长度等问题。

联网检索同样属于完整复现的重要依赖。代码脚本只能构造检索词和处理结果，真正的公开政策、学院细则、培养方案与历史数据仍由平台搜索工具提供。因此 README 将“Python 标准库依赖”和“平台工具依赖”分开说明：前者只需要 Python 3.x 及 re 模块，后者包括大模型服务、聚合搜索、分支器、变量存储/文本处理以及工作流运行环境。

完成上述节点、参数和变量配置后，应按前述测试顺序逐类验证；所有基础场景通过后，再使用十组核心测试用例进行完整回归。

十四、后续扩展路线

（一）短期迭代

1. 进一步完善学校简称到官方全称的映射，高频唯一简称自动标准化，存在歧义时要求用户确认。

2. 加强学院级与专业级历史推免数据检索，减少只能使用学校整体比例的情况。

3. 增加年份权重和来源域名优先级，最新官方文件优先。

4. 继续扩展测试集，覆盖不同专业、不同校区、专项计划、学院政策冲突等复杂情形。

5. 优化最终报告冗余度，在保证各项内容完整的前提下提高阅读效率。

短期迭代以稳定性、数据准确性和边界覆盖为核心，所有修改均需通过固定测试集回归后才能合并到主版本。

（二）中长期扩展

中长期计划建立“学校—学院—专业—届次”四层政策索引，记录文件名称、来源、年份、适用对象、关键规则和更新时间。这样可以显著减少重复检索，提高政策信息的获取效率和准确性。四层索引的设计思路是：学校层记录推免名额总量和基本资格条件；学院层记录实施细则和加分规则；专业层记录培养方案和专业特殊要求；届次层记录不同年份的政策变化。通过这种分层结构，系统可以快速定位到学生所需的具体政策文件，而不需要每次都重新进行全网检索。同时，政策索引还会记录每条信息的更新时间和来源链接，确保学生能够追溯到原始官方文件，提高信息的可信度和可验证性。

同时可增加学生成长档案：每学期保存排名、GPA、英语、竞赛科研变化，生成趋势分析。大三阶段进一步加入夏令营截止日期、材料状态、导师联系、面试准备和目标院校申请进度，使系统从一次性评估升级为持续升学规划助手。

无论功能如何扩展，系统仍坚持三个底线：重要资格以高校官方公示为准；无法确认的信息必须明确标注不确定；不同数据口径不得混用。

十五、总结

“推免智策——大学生保研资格评估与升学规划智能体”围绕本科生在推免准备过程中最常遇到的三个问题展开：第一，学校、学院和专业政策分散，学生很难快速判断哪些规则真正适用于自己；第二，推免名额、报名门槛、研究生接收人数、历史保研率等数据口径复杂，稍有混淆就会得出完全不同的结论；第三，不同年级学生面对的任务不同，大一、大二需要长期成长路径，大三则必须同时完成本校资格确认和目标院校申请。针对这些问题，项目没有把智能体设计成单一的“问一句、答一段”聊天机器人，而是搭建了从资料读取、基础校验、联网检索、学校资格核验、政策解析、结果标准化、条件分支到最终报告生成的完整工作流，使每一步都有清晰输入、输出和异常处理。

在技术实现上，项目经历了多轮真实调试和结构迭代。最初版本更多依赖大模型和 Agent 自主完成检索与判断，但在实际运行中出现过 ReACT 推理格式错误、插件名称协议异常、搜索正文过长触发上下文问题、不同学校推免率被错误趋同、真实高校因学院信息缺失被误判为学校核验失败，以及最终长报告因为 maxTokens 不足而中途截断等问题。团队没有通过不断堆叠提示词来掩盖这些故障，而是逐步把确定性的任务下沉到 Python 代码节点：排名和人数关系由代码检查，检索词由脚本稳定生成，学校真实性和本科推免资格由标准化 Boolean 控制；联网搜索独立为工具节点，普通大模型只负责解析搜索结果；最终报告节点则专注于分年级分析和自然语言表达。通过这种“模型负责理解、代码负责规则、工具负责证据、分支器负责流程”的组合，系统的可复现性和可解释性明显提高。

在结果设计上，项目坚持“先核验、再评估、后规划”。学校真实且具有本科推免资格是进入完整报告的前提，但学院细则、专业推免率或个人英语成绩暂时缺失时，只会被记录为不确定信息，不会反向把学校判定失败；同时，系统明确区分本科推荐名额与研究生接收推免生人数，也不把“排名前 X%可申请”直接等同于真实推免率。在无法获取专业级官方比例时，系统按专业、学院、学校整体和多来源辅助数据逐级降级，并在报告中说明数据层级与可靠程度。大一、大二报告重点提供排名目标、英语、竞赛科研和分阶段计划；大三报告则在本校资格评估基础上保留冲刺、匹配、相对稳妥三档院

校建议，并给出夏令营、预推免、九推及材料准备时间线。所有机会区间和院校定位均属于阶段性规划参考，不构成资格或录取承诺。

复赛阶段，团队进一步建立了固定测试集和回归机制。最新 10 组测试覆盖合肥工业大学、清华大学、中国科学技术大学等常用高校，包含大一、大二、大三正常流程、HFUT 和中文简称、缺少专业、缺少排名、排名大于总人数、虚构高校以及无推免资格高校等情况。每次修改工作流后，都必须重新运行正常样例和历史故障样例，确认修复没有破坏已有功能。与此同时，README 对 Python 标准库依赖、星辰 Agent 平台依赖、大模型与聚合搜索工具依赖进行分别说明，使评审人员能够理解三份脚本与完整智能体之间的关系。通过这些措施，项目已经从“能够生成一份保研报告”逐步发展为“能够核验信息、控制流程、解释依据、处理异常并持续回归测试”的工程化智能体。

后续迭代将继续围绕数据时效性和学院级精细度展开：一方面完善高校简称与官方全称映射、政策年份优先级和学院专业归属识别；另一方面建立按学校—学院—专业—届次组织的政策索引，减少每次从零搜索的成本。长期还可加入学生成长档案、学期排名变化、英语和成果积累趋势、目标院校申请清单与时间节点管理，使系统从一次性的“当前评估”扩展为贯穿本科阶段的持续升学规划助手。无论功能如何增加，项目仍将坚持官方来源优先、数据口径清晰、未知信息透明和最终资格以高校正式公示为准的基本原则。

从复赛评审视角看，本项目的价值还体现在可复现的工程过程上：每一次异常都对应明确的节点、输入和修复措施，三份脚本能够单独检查，十组测试数据能够重复执行，联网信息能够区分官方与非官方来源，最终报告则把未知条件集中到不确定性说明中。团队后续不会把“更复杂”简单等同于“更智能”，而是继续以准确、稳定、可解释和可执行为迭代优先级，使推免智策真正服务于学生长期升学规划。

展望未来，项目将继续秉持准确、稳定、可解释、可执行的四项核心原则，持续推进功能迭代和质量提升。在准确性方面，将持续完善学校名称标准化、政策来源可信度评估和数据口径识别能力，确保每一条输出都有可靠的依据支撑。在稳定性方面，将进一步优化异常处理机制，提高系统在各种边界条件下的容错能力，减少因输入变化导致的输出波动。在可解释性方面，将继续完善报告结构，使每一条建议都能追溯到具体的政策依据或数据支撑，增强用户对系统输出的信任度。在可执行性方面，将把规划建议

进一步拆解为可检查、可追踪的任务清单，帮助学生将“知道该做什么”转化为“正在做什么”。